

Calcolatori Elettronici T

Ingegneria Informatica

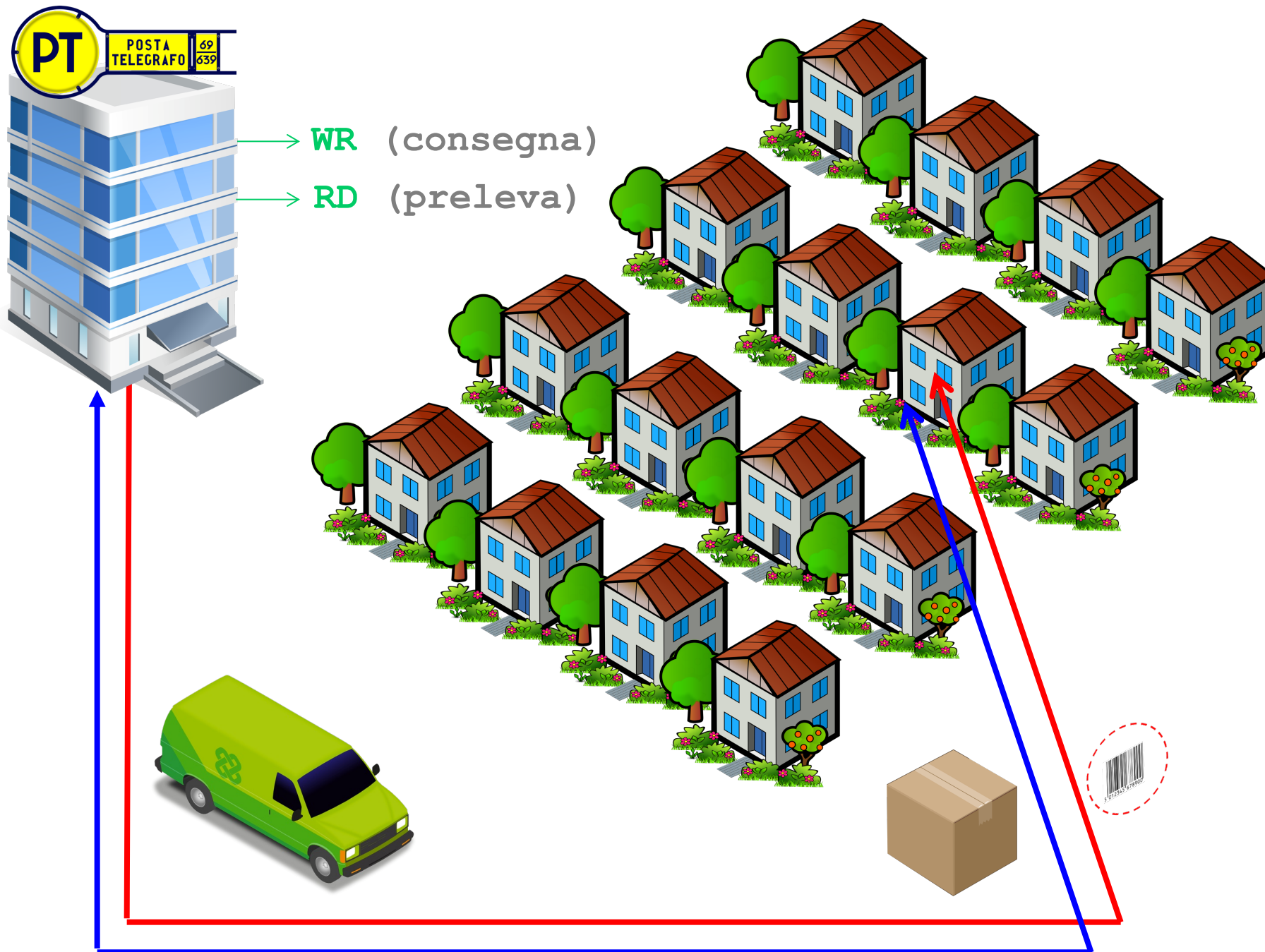
02 Mapping e decodifica



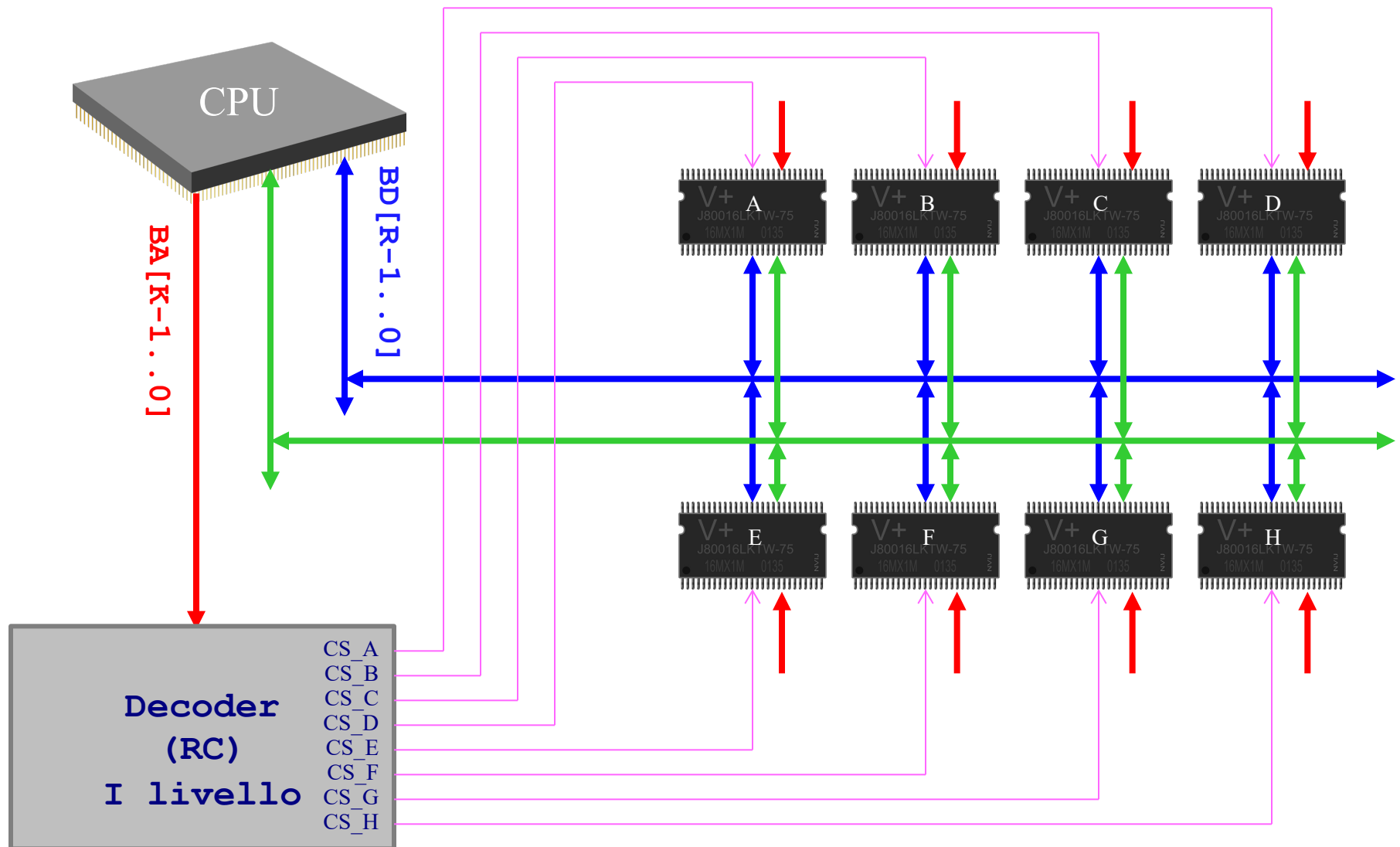
Spazio di indirizzamento

- Una CPU emette un certo numero di indirizzi e altri segnali sui bus di sistema per poter comunicare con altri moduli
- Il numero di diversi indirizzi emessi dalla CPU costituisce lo spazio di indirizzamento
- Una CPU che emette un indirizzo a 20 bit ha uno spazio di indirizzamento di 1 MB (2^{20} differenti indirizzi)
- Una CPU che emette un indirizzo a 32 bit ha uno spazio di indirizzamento di 4 GB (2^{32} differenti indirizzi)
- Le prime CPU avevano spazi di indirizzamento molto ridotto, di alcuni KB (e.g., 64 KB o meno)
- Oggi è consuetudine avere almeno 32 bit di indirizzo

Un indirizzo per distribuire merci

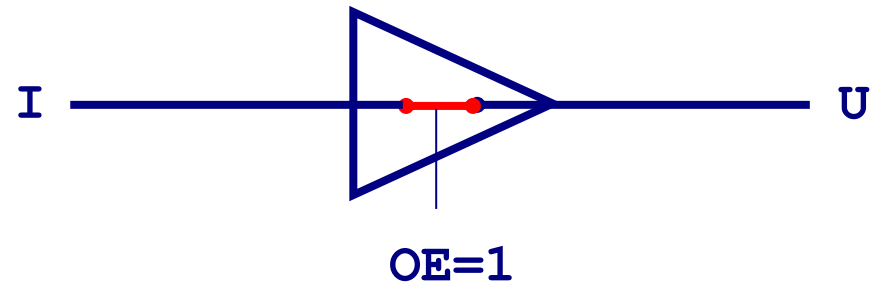
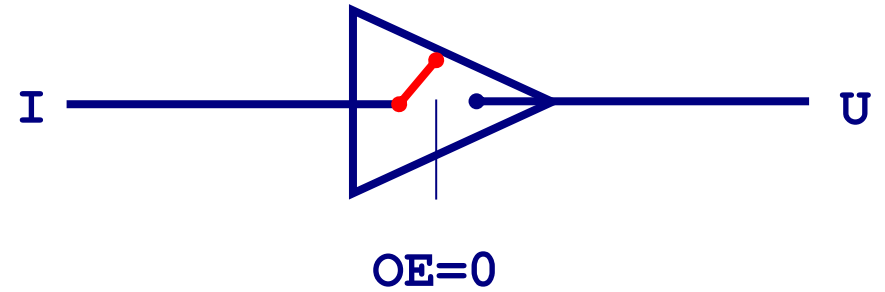
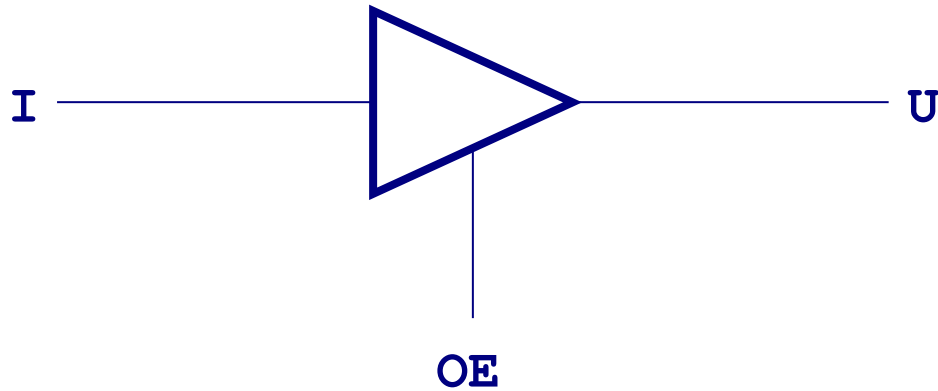


Un indirizzo per distribuire dati (CPU)

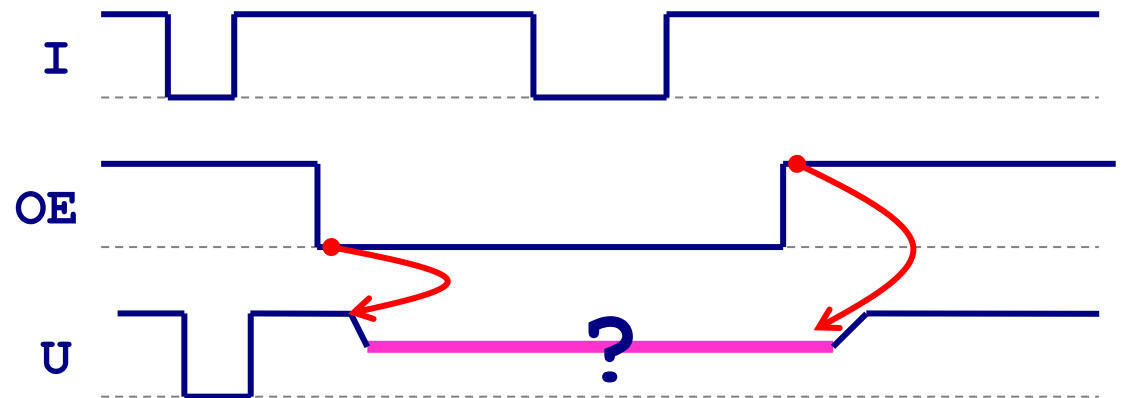


Il decoder di II livello
è all'interno di ciascun
dispositivo (memoria, etc)

Driver 3-state

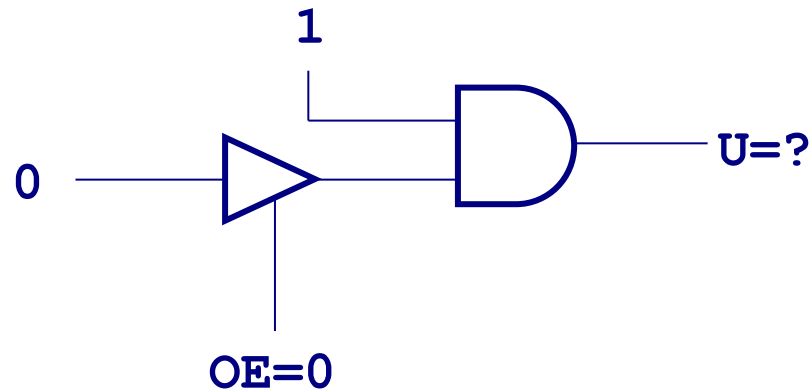


OE	I	U
1	0	0
1	1	1
0	0	Z
0	1	Z



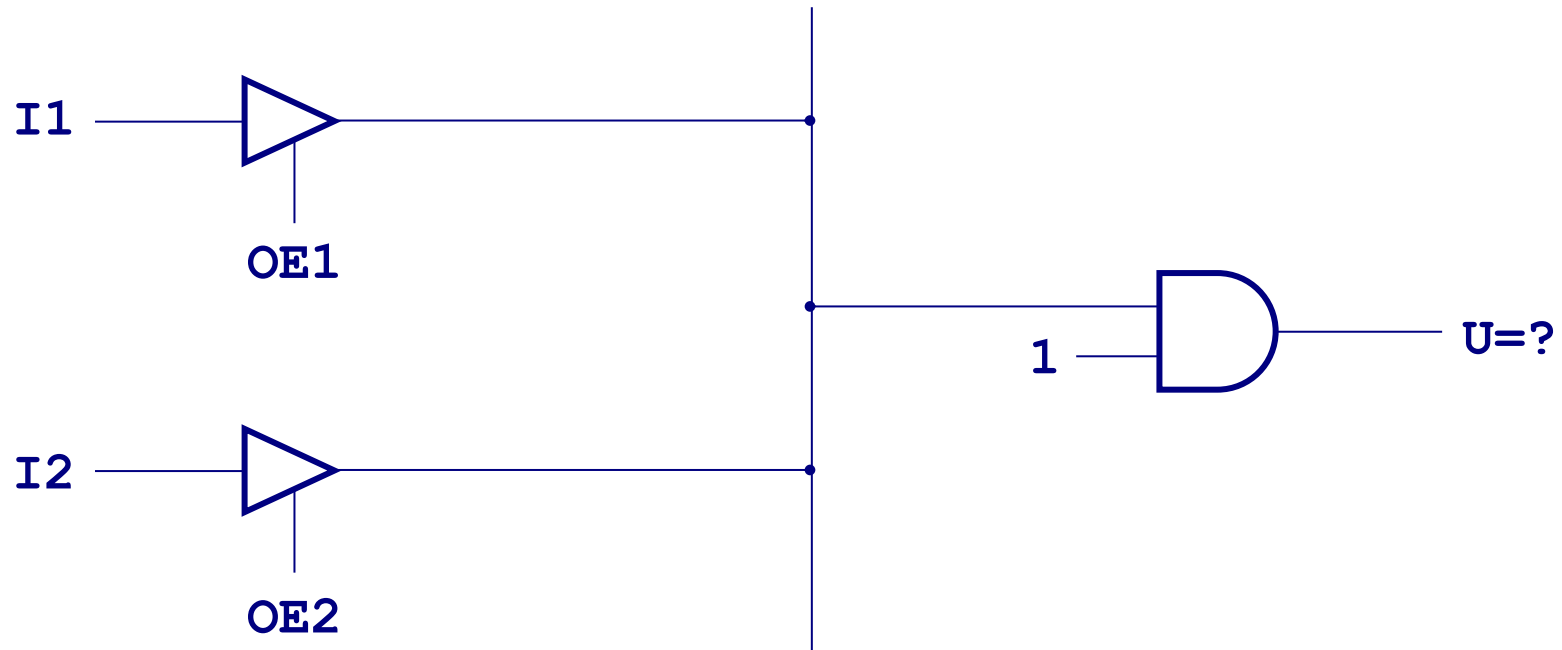
Quale è il valore della tensione ?

Quale valore logico assume U ?



Che cosa è necessario garantire nella rete seguente ?

Quando il segnale U assume un valore logico significativo ?

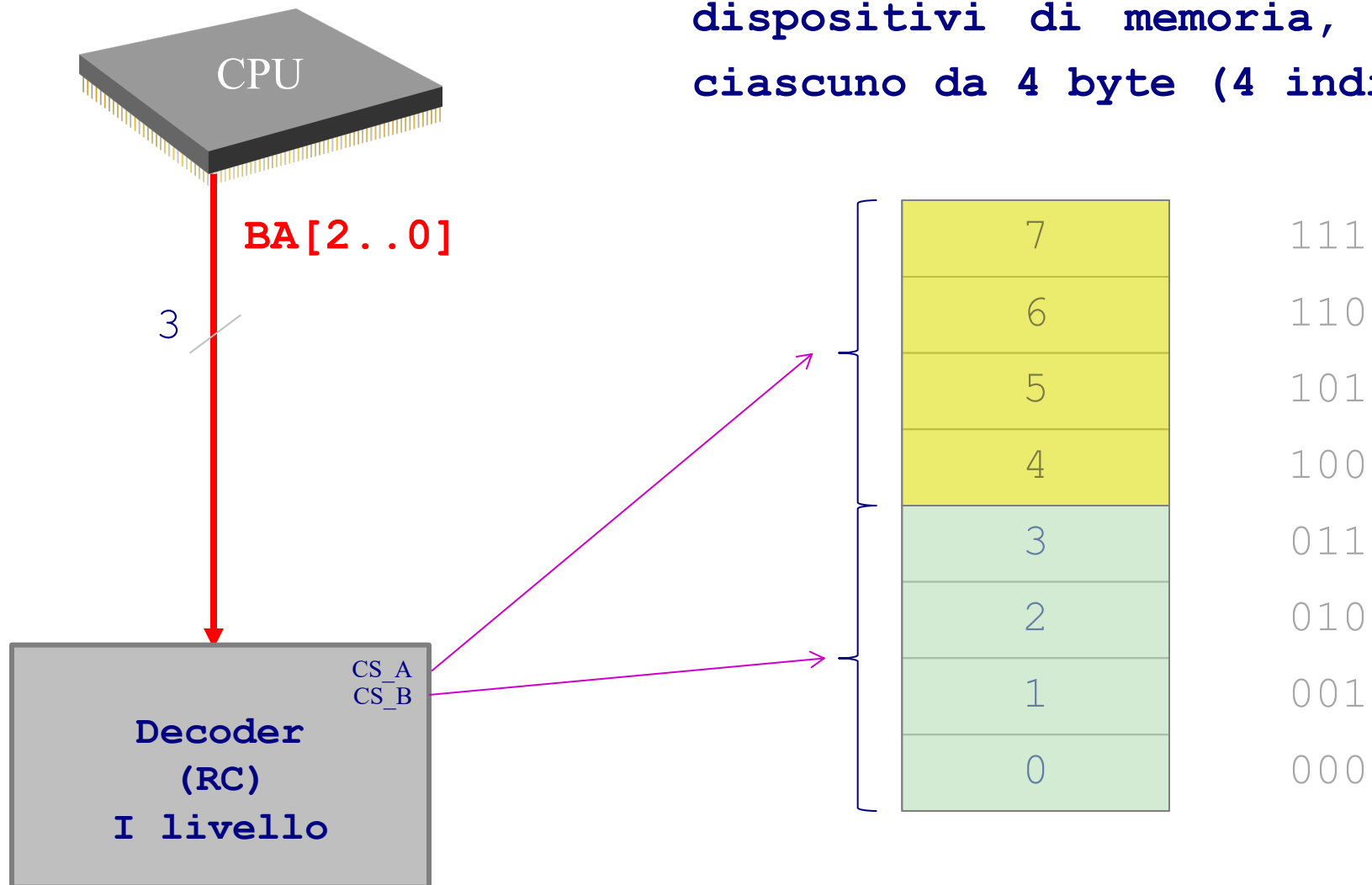


Condizione di visibilità di un dispositivo da parte del software

- Condizione necessaria affinché un dispositivo fisico (memoria, interfaccia, o altra entità) sia accessibile al software è:
 - *il dispositivo deve essere mappato in uno spazio di indirizzamento*
- Mappare in uno spazio di indirizzamento significa:
 - *associare al dispositivo una finestra di indirizzi di quello spazio di indirizzamento*
- La CPU comunica con i dispositivi mappati nello spazio di indirizzamento unicamente attraverso dei cicli di bus
- A ciascun indirizzo è associato un byte di informazione

Esempio: una CPU con K=3 bit di indirizzo

- Lo spazio di indirizzamento sarebbe di solo 8 elementi
- Supponiamo di avere solo due dispositivi di memoria, A e B, ciascuno da 4 byte (4 indirizzi)



- Come facciamo ad attivare una delle due memorie in base all'indirizzo emesso dalla CPU sui **BA[2..0]**?
- Ovvero, come è fatta la rete di decodifica (I livello) che genera i due segnali CS_A e CS_B?

CS_A = **BA2**

CS_B = **BA2***

- Questi segnali saranno inviati a alle memorie (decodifica di I livello)
- Poi, sarà individuato l'elemento all'interno delle memoria selezionata (decodifica di II livello)

BA2=1

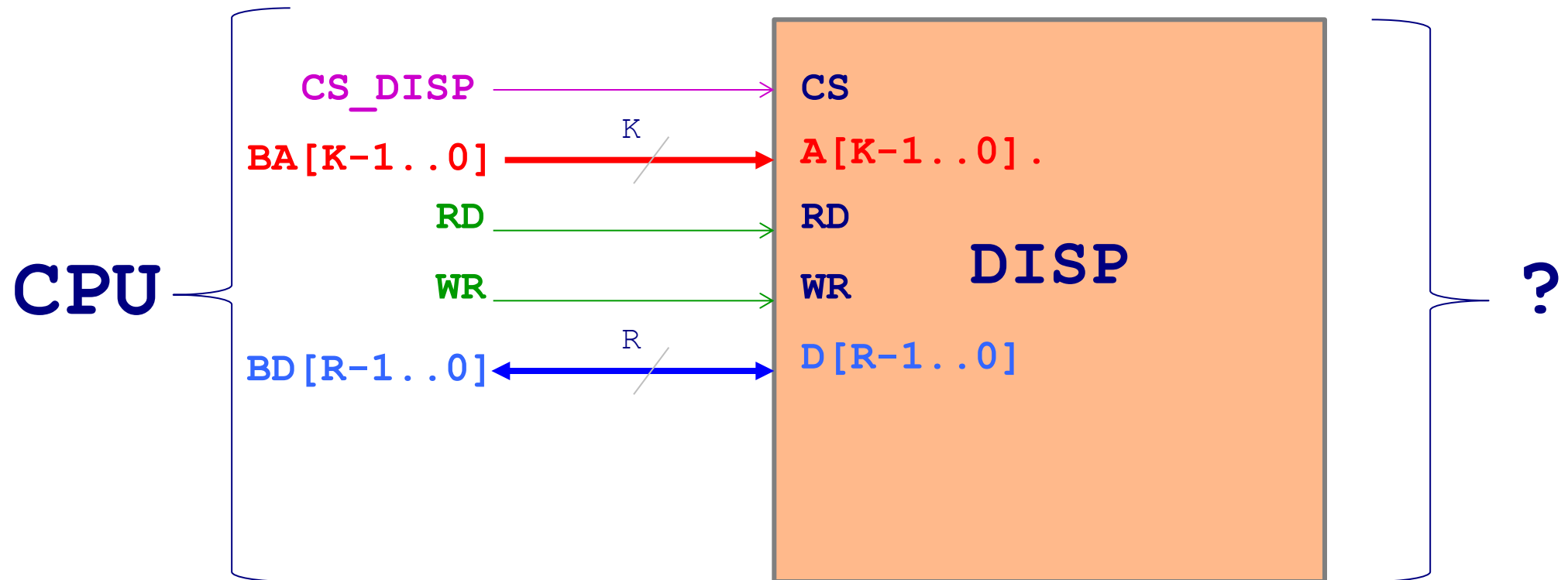
BA2=0

{	7	111
	6	110
	5	101
	4	100
{	3	011
	2	010
	1	001
	0	000

↑
II livello

Come è fatto un generico dispositivo?

- Un qualsiasi dispositivo (memoria, periferica, etc), comunica con la CPU mediante una interfaccia standard a sx



- La comunicazione con l'esterno avviene secondo modalità che sono specifiche del dispositivo e quindi non standard
- BA[K-1..0]** utilizzati (internamente) per la decodifica di II livello

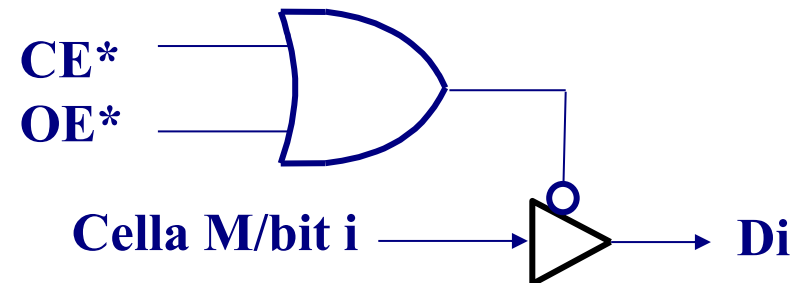
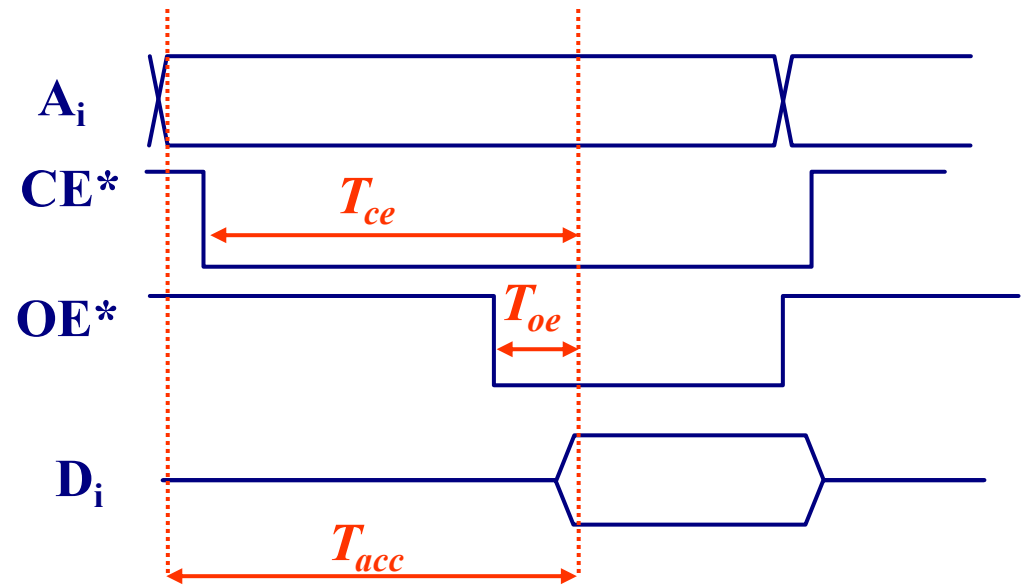
Memorie EPROM

EPROM

1	VPP	VCC	32
2	A16	PGM*	31
3	A15	NC	30
4	A12	A14	29
5	A7	A13	28
6	A6	A8	27
7	A5	A9	26
8	A4	A11	25
9	A3	OE*	24
10	A2	A10	23
11	A1	CE*	22
12	A0	D7	21
13	D0	D6	20
14	D1	D5	19
15	D2	D4	18
16	GND	D3	17

128K × 8

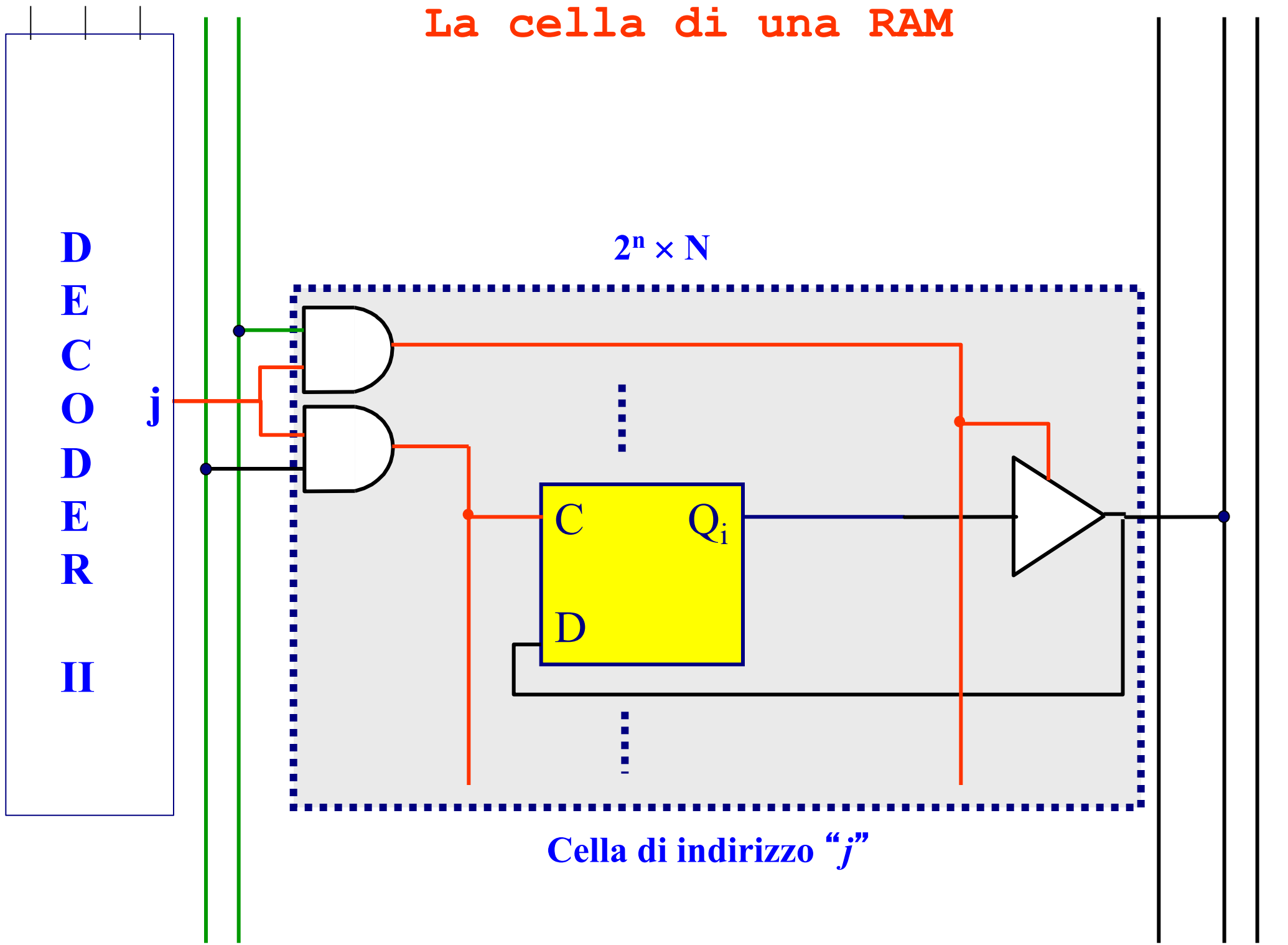
- Memorie non volatili a sola lettura
- Capacità a multipli di 2:
32K, 64K, 128K, 256K, etc



A_0 A_1 A_{n-1} WR RD

D_0 D_i D_{N-1}

La cella di una RAM



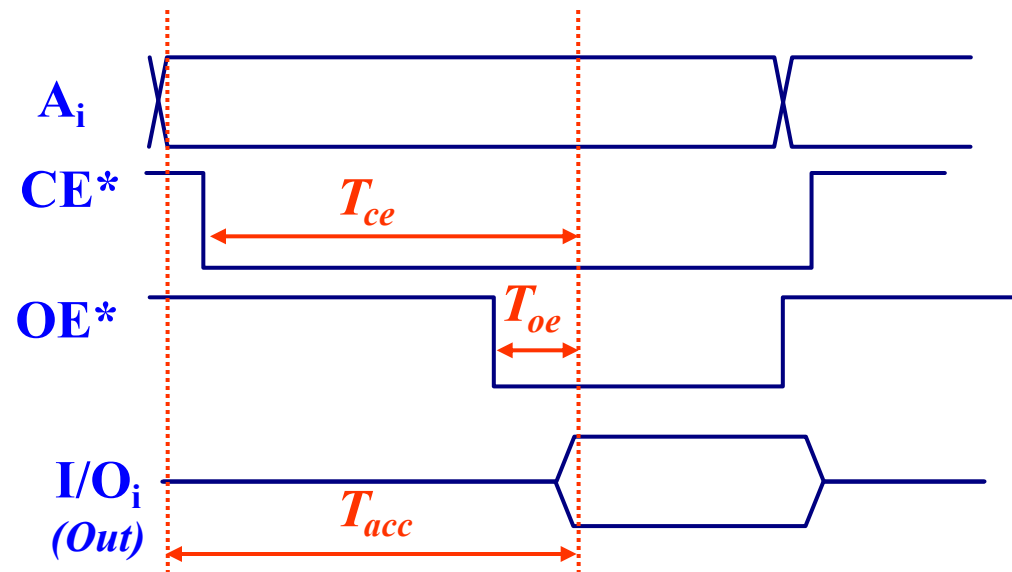
Memorie RAM (SRAM)

RAM

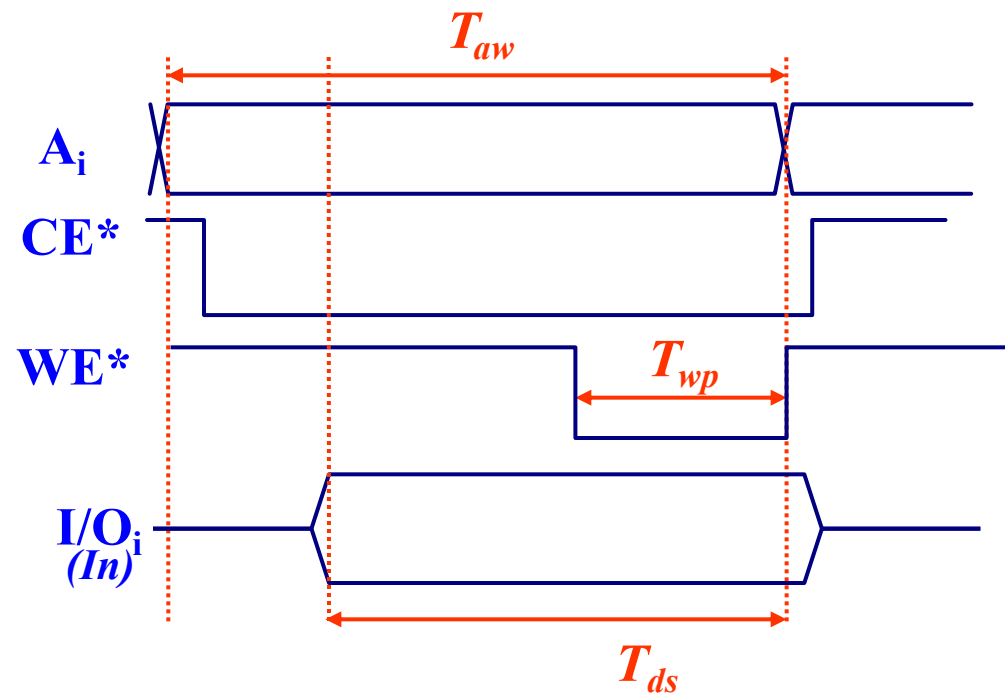
1	NC	VCC	32
2	A16	A15	31
3	A14	NC	30
4	A12	WE*	29
5	A7	A13	28
6	A6	A8	27
7	A5	A9	26
8	A4	A11	25
9	A3	OE*	24
10	A2	A10	23
11	A1	CE*	22
12	A0	I/O7	21
13	I/O0	I/O6	20
14	I/O1	I/O5	19
15	I/O2	I/O4	18
16	GND	I/O3	17

128K × 8

Read Cycle



Write Cycle



- Memorie volatili, leggibili e scrivibili
- Capacità a multipli di 4: 8K, 32K, 128K, 512K, etc
- DRAM: 1 transistor per bit, maggiore capacità, più lente



CY7C199

32K x 8 Static RAM

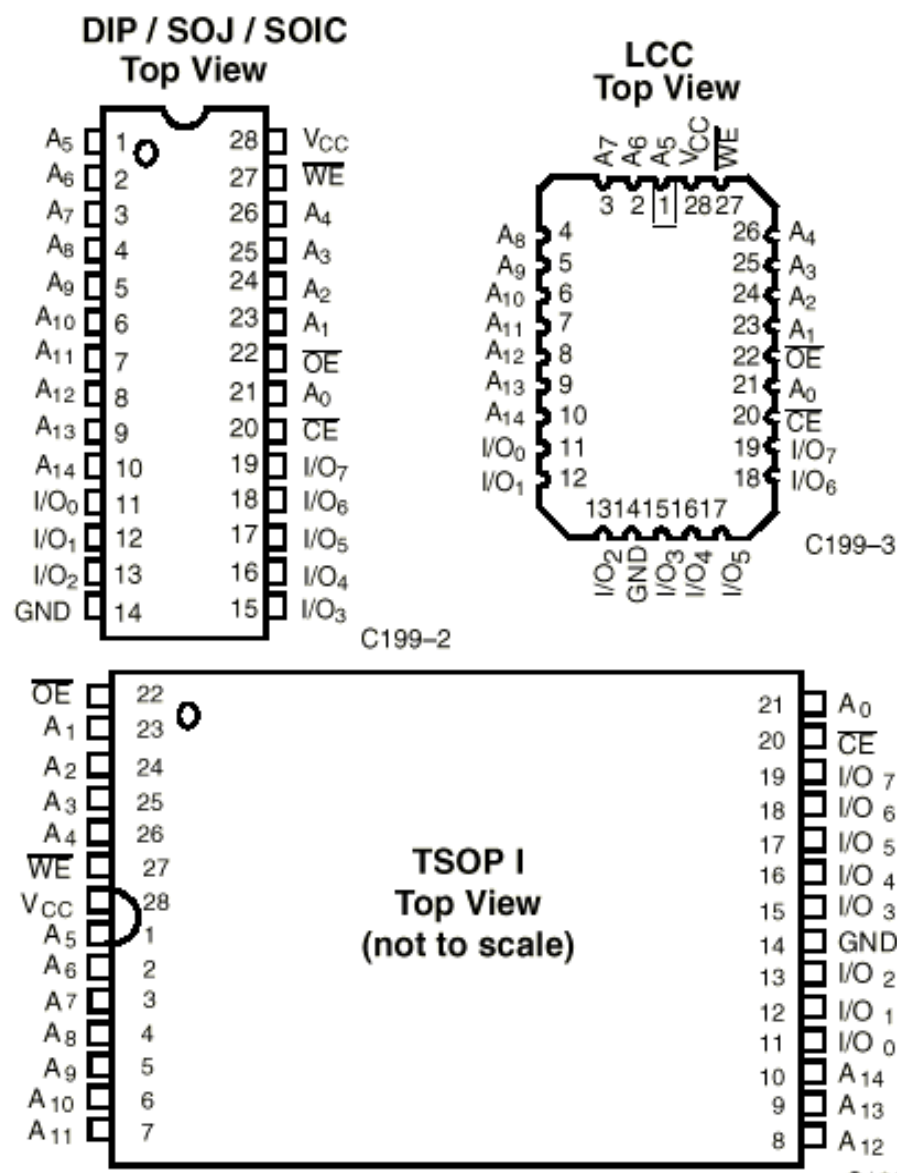
Functional Description

The CY7C199 is a high-performance CMOS static RAM organized as 32,768 words by 8 bits. Easy memory expansion is provided by an active LOW chip enable ($\overline{CE}$) and active LOW output enable ($\overline{OE}$) and three-state drivers. This device has an automatic power-down feature, reducing the power consumption by 81% when deselected. The CY7C199 is in the standard 300-mil-wide DIP, SOJ, and LCC packages.

An active LOW write enable signal ($\overline{WE}$) controls the writing/reading operation of the memory. When $\overline{CE}$ and $\overline{WE}$ inputs are both LOW, data on the eight data input/output pins (I/O₀ through I/O₇) is written into the memory location addressed by the address present on the address pins (A₀ through A₁₄). Reading the device is accomplished by selecting the device and enabling the outputs, $\overline{CE}$ and $\overline{OE}$ active LOW, while $\overline{WE}$ remains inactive or HIGH. Under these conditions, the contents of the location addressed by the information on address pins are present on the eight data input/output pins.

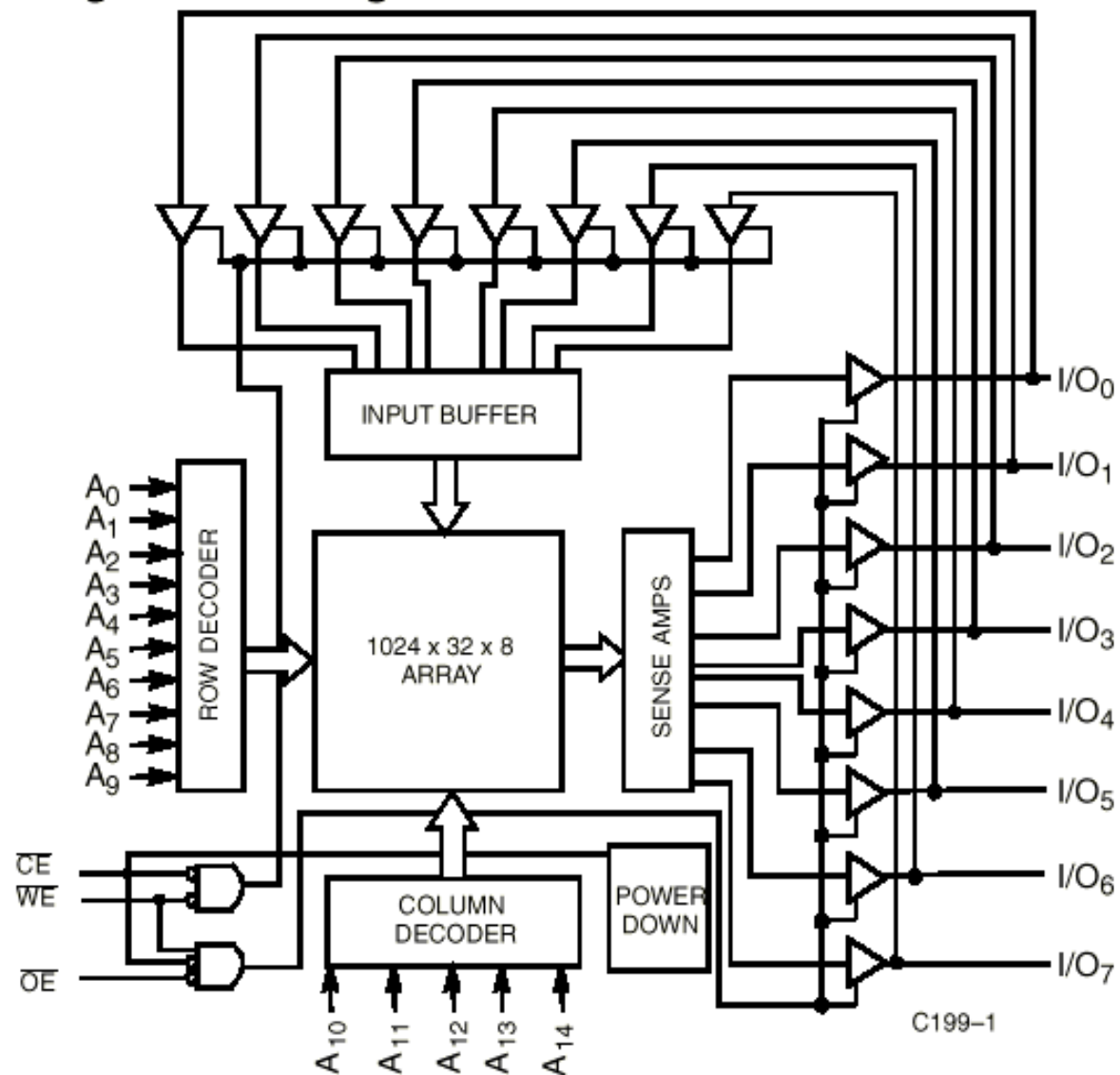
The input/output pins remain in a high-impedance state unless the chip is selected, outputs are enabled, and write enable ($\overline{WE}$) is HIGH. A die coat is used to improve alpha immunity.

Pin Configurations





Logic Block Diagram



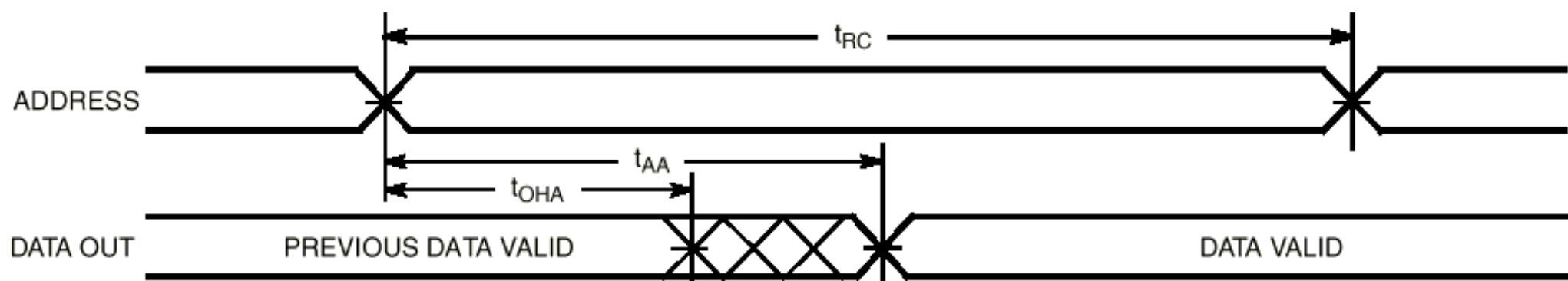


CYPRESS

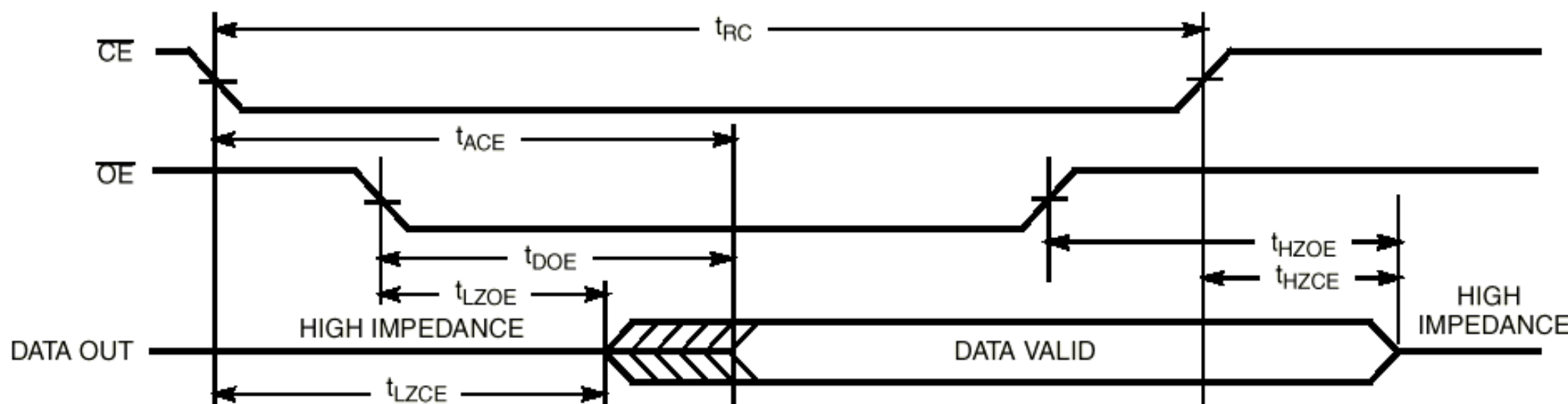
CY7C199

32K x 8 Static RAM

Read Cycle No. 1^[12, 13]



Read Cycle No. 2^[13, 14]



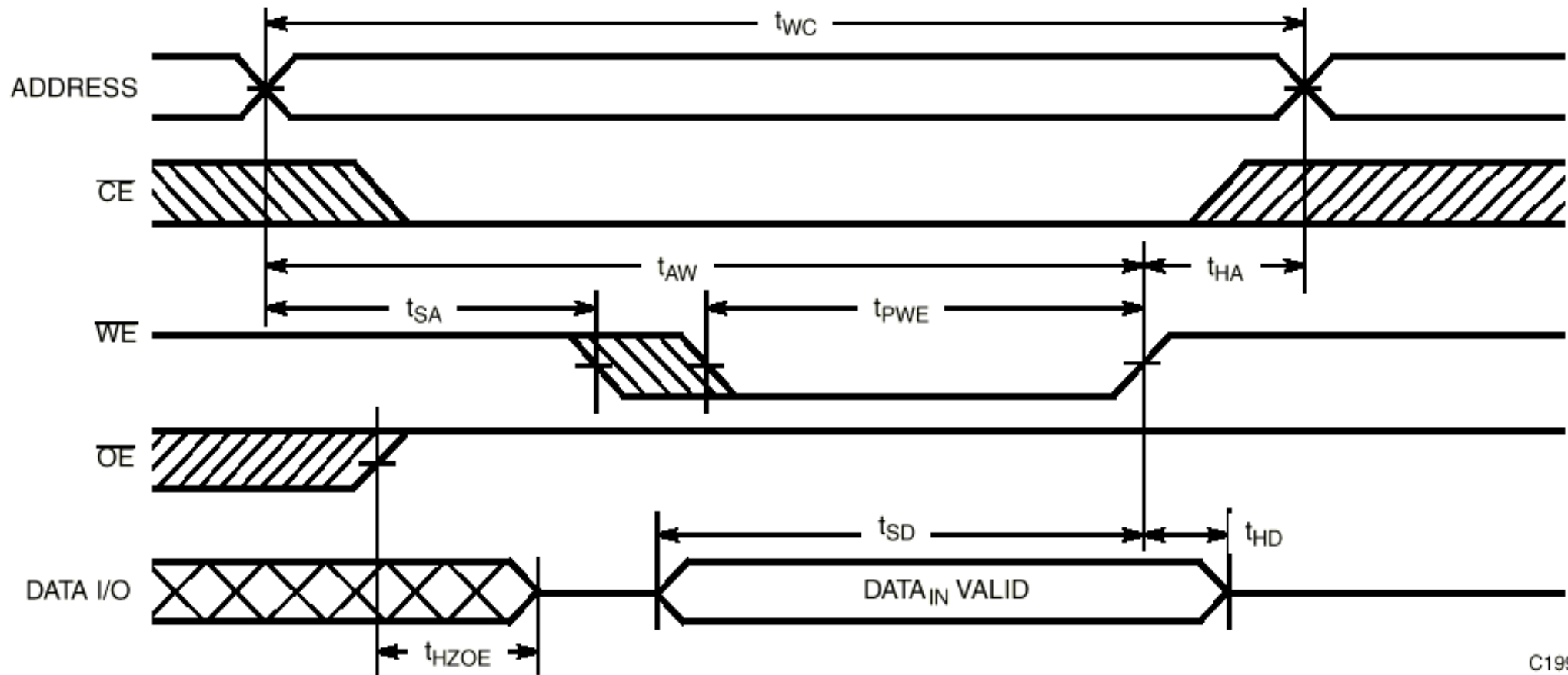


CYPRESS

CY7C199

32K x 8 Static RAM

Write Cycle No. 1 (WE Controlled)^[10, 15, 16]



C199

Switching Characteristics Over the Operating Range^[3,7] (continued)

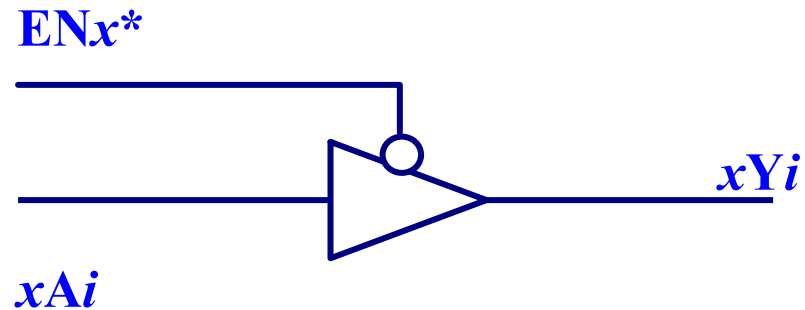
Parameter	Description	7C199-20		7C199-25		7C199-35		7C199-45		Unit
		Min.	Max.	Min.	Max.	Min.	Max.	Min.	Max.	
READ CYCLE										
t _{RC}	Read Cycle Time	20		25		35		45		ns
t _{AA}	Address to Data Valid		20		25		35		45	ns
t _{OHA}	Data Hold from Address Change	3		3		3		3		ns
t _{ACE}	$\overline{CE}$ LOW to Data Valid		20		25		35		45	ns
t _{DOE}	$\overline{OE}$ LOW to Data Valid		9		10		16		16	ns
t _{LZOE}	$\overline{OE}$ LOW to Low Z ^[8]	0		0		0		0		ns
t _{HZOE}	$\overline{OE}$ HIGH to High Z ^[8,9]		9		11		15		15	ns
t _{LZCE}	$\overline{CE}$ LOW to Low Z ^[8]	3		3		3		3		ns
t _{HZCE}	$\overline{CE}$ HIGH to High Z ^[8,9]		9		11		15		15	ns
t _{PU}	$\overline{CE}$ LOW to Power-Up	0		0		0		0		ns
t _{PD}	$\overline{CE}$ HIGH to Power-Down		20		20		20		25	ns
WRITE CYCLE ^[10,11]										
t _{WC}	Write Cycle Time	20		25		35		45		ns
t _{SCE}	$\overline{CE}$ LOW to Write End	15		18		22		22		ns
t _{AW}	Address Set-Up to Write End	15		20		30		40		ns
t _{HA}	Address Hold from Write End	0		0		0		0		ns
t _{SA}	Address Set-Up to Write Start	0		0		0		0		ns
t _{PWE}	$\overline{WE}$ Pulse Width	15		18		22		22		ns
t _{SD}	Data Set-Up to Write End	10		10		15		15		ns
t _{HD}	Data Hold from Write End	0		0		0		0		ns
t _{HZWE}	$\overline{WE}$ LOW to High Z ^[9]		10		11		15		15	ns
t _{LZWE}	$\overline{WE}$ HIGH to Low Z ^[8]	3		3		3		3		ns

Integrati Notevoli: “244”

74XX244

1A1	1Y1
1A2	1Y2
1A3	1Y3
1A4	1Y4
2A1	2Y1
2A2	2Y2
2A3	2Y3
2A4	2Y4
EN1*	EN2*

Driver 3-state ad 8-bit
(strutturato in 2 gruppi di 4 bit)



SN54AC244, SN74AC244 OCTAL BUFFERS/DRIVERS WITH 3-STATE OUTPUTS

SCAS514C – JUNE 1995 – REVISED SEPTEMBER 1996

description

These octal buffers and line drivers are designed specifically to improve the performance and density of 3-state memory address drivers, clock drivers, and bus-oriented receivers and transmitters.

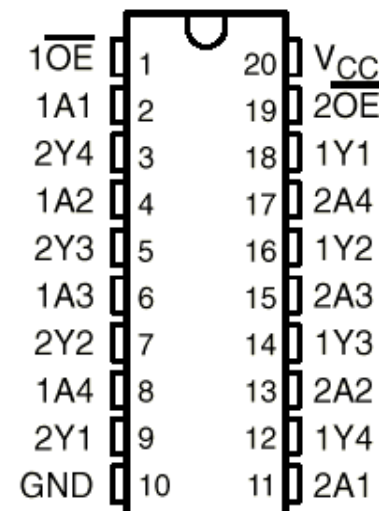
The 'AC244 are organized as two 4-bit buffers/drivers with separate output-enable ($\overline{OE}$) inputs. When $\overline{OE}$ is low, the device passes noninverted data from the A inputs to the Y outputs. When $\overline{OE}$ is high, the outputs are in the high-impedance state.

The SN54AC244 is characterized for operation over the full military temperature range of -55°C to 125°C . The SN74AC244 is characterized for operation from -40°C to 85°C .

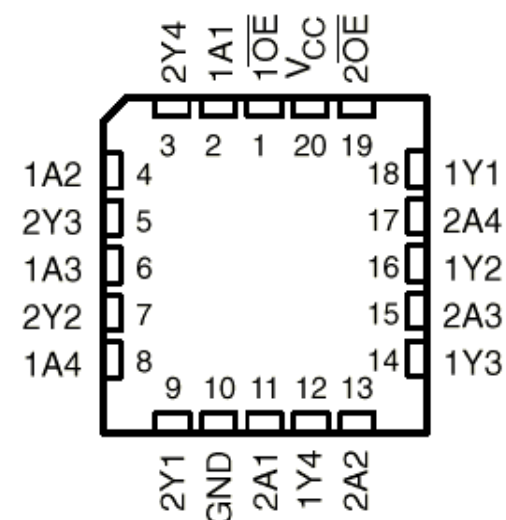
FUNCTION TABLE
(each buffer)

INPUTS		OUTPUT Y
$\overline{OE}$	A	
L	H	H
L	L	L
H	X	Z

SN54AC244 . . . J OR W PACKAGE
SN74AC244 . . . DB, DW, N, OR PW PACKAGE
(TOP VIEW)



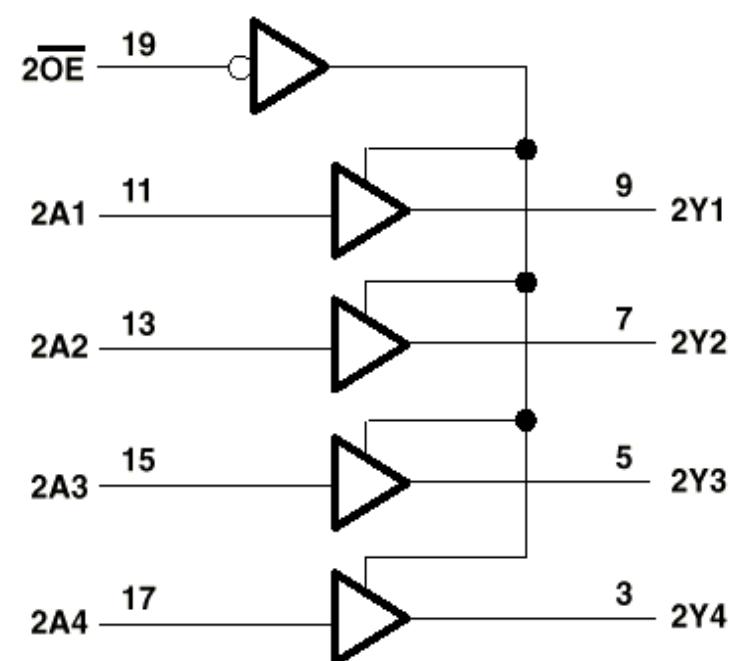
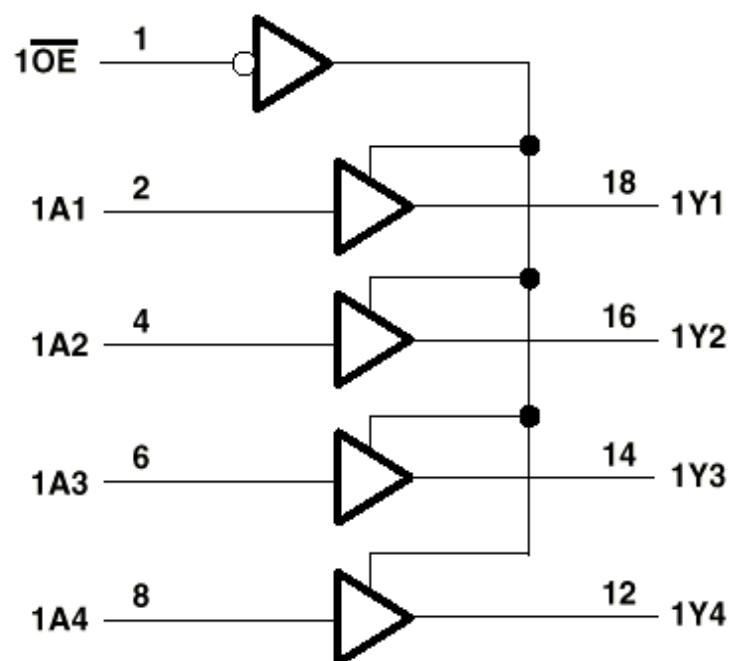
SN54AC244 . . . FK PACKAGE
(TOP VIEW)



SN54AC244, SN74AC244 OCTAL BUFFERS/DRIVERS WITH 3-STATE OUTPUTS

SCAS514C – JUNE 1995 – REVISED SEPTEMBER 1996

logic diagram (positive Logic)



SN54AC244, SN74AC244 OCTAL BUFFERS/DRIVERS WITH 3-STATE OUTPUTS

SCAS514C – JUNE 1995 – REVISED SEPTEMBER 1996

switching characteristics over recommended operating free-air temperature range,
 $V_{CC} = 5\text{ V} \pm 0.5\text{ V}$ (unless otherwise noted) (see Figure 1)

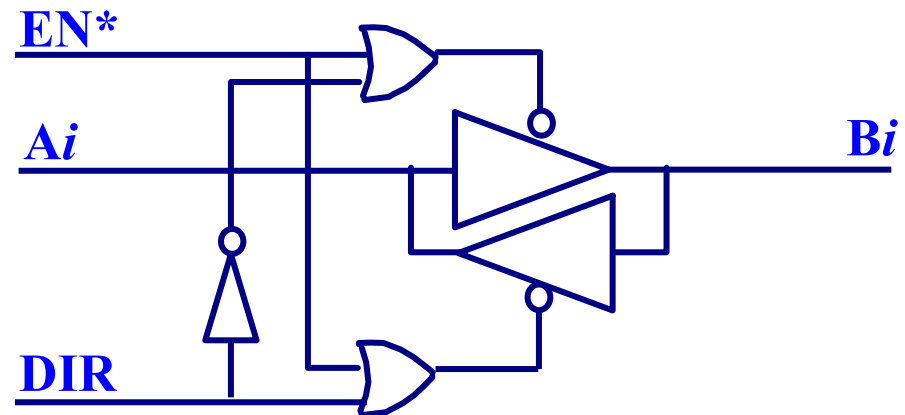
PARAMETER	FROM (INPUT)	TO (OUTPUT)	$T_A = 25^\circ\text{C}$			SN54AC244		SN74AC244		UNIT
			MIN	TYP	MAX	MIN	MAX	MIN	MAX	
t_{PLH}	A	Y	1.5	5	7	1	9.5	1	7.5	ns
t_{PHL}			1.5	5	7	1	9	1	7.5	
t_{PZH}	$\overline{OE}$	Y	1.5	5	7	1	9	1.5	8	ns
t_{PZL}			1.5	5.5	8	1	10.5	1.5	8.5	
t_{PHZ}	$\overline{OE}$	Y	2.5	6.5	9	1	10.5	1	9.5	ns
t_{PLZ}			2	6.5	9	1	11	2	9.5	

Integrati Notevoli: “245”

74XX245

A1	B1
A2	B2
A3	B3
A4	B4
A5	B5
A6	B6
A7	B7
A8	B8
EN*	DIR

Driver bidirezionale (*transceiver*)
ad 8-bit.



74AC11245

OCTAL BUS TRANSCEIVER WITH 3-STATE OUTPUTS

SCAS010B – JULY 1987 – REVISED APRIL 1996

description

This octal bus transceiver is designed for asynchronous two-way communication between data buses. The control-function implementation minimizes external timing requirements.

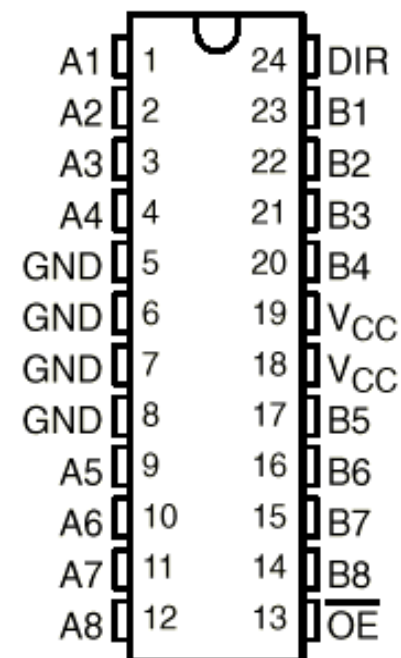
The device allows noninverted data transmission from the A bus to the B bus or from the B bus to the A bus, depending on the logic level at the direction-control (DIR) input. The output-enable ($\overline{\text{OE}}$) input can be used to disable the device so that the buses are effectively isolated.

The 74AC11245 is characterized for operation from -40°C to 85°C .

FUNCTION TABLE

OUTPUT ENABLE $\overline{\text{OE}}$	DIRECTION CONTROL DIR	OPERATION
L	L	B data to A bus
L	H	A data to B bus
H	X	Isolation

DB, DW, NT, OR PW PACKAGE
(TOP VIEW)



SCAS010B – JULY 1987 – REVISED APRIL 1996

74AC11245
OCTAL BUS TRANSCEIVER
WITH 3-STATE OUTPUTS
 SCAS010B – JULY 1987 – REVISED APRIL 1996

switching characteristics over recommended operating free-air temperature range,
 $V_{CC} = 5\text{ V} \pm 0.5\text{ V}$ (unless otherwise noted) (see Figure 1)

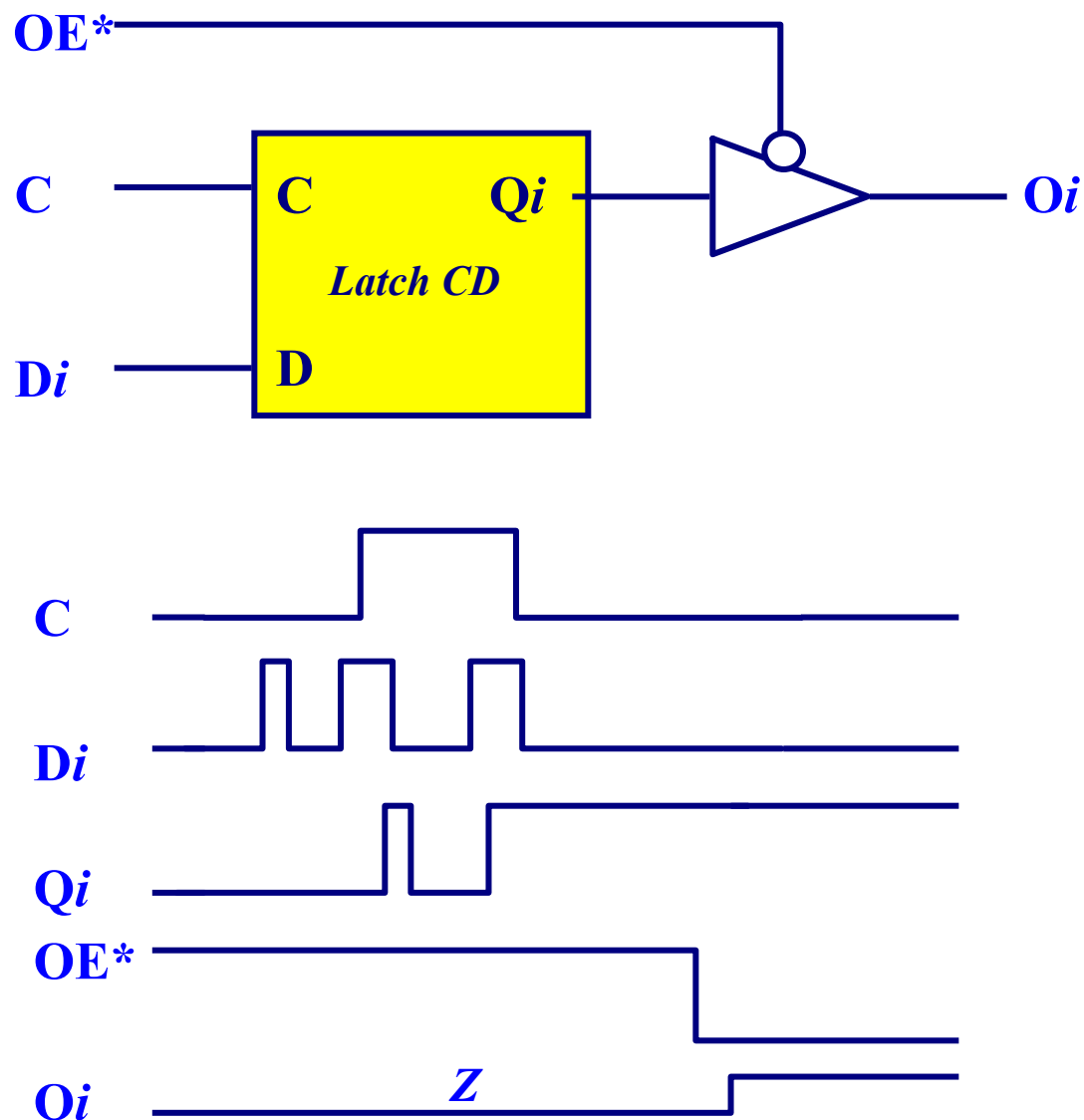
PARAMETER	FROM (INPUT)	TO (OUTPUT)	$T_A = 25^\circ\text{C}$			MIN	MAX	UNIT
			MIN	TYP	MAX			
t_{PLH}	A or B	B or A	1.5	4.8	8.5	1.5	9.5	ns
t_{PHL}			1.5	4.1	6.3	1.5	6.9	
t_{PZH}	$\overline{OE}$	B or A	1.5	6.2	10.2	1.5	11.4	ns
t_{PZL}			1.5	5.9	8.6	1.5	9.5	
t_{PHZ}	$\overline{OE}$	B or A	1.5	6.4	8.8	1.5	9.5	ns
t_{PLZ}			1.5	7	9.6	1.5	10.4	

Integrati Notevoli: "373"

74XX373

D0	O0
D1	O1
D2	O2
D3	O3
D4	O4
D5	O5
D6	O6
D7	O7
C	OE*

Latch a 8-bit
con uscite 3-state

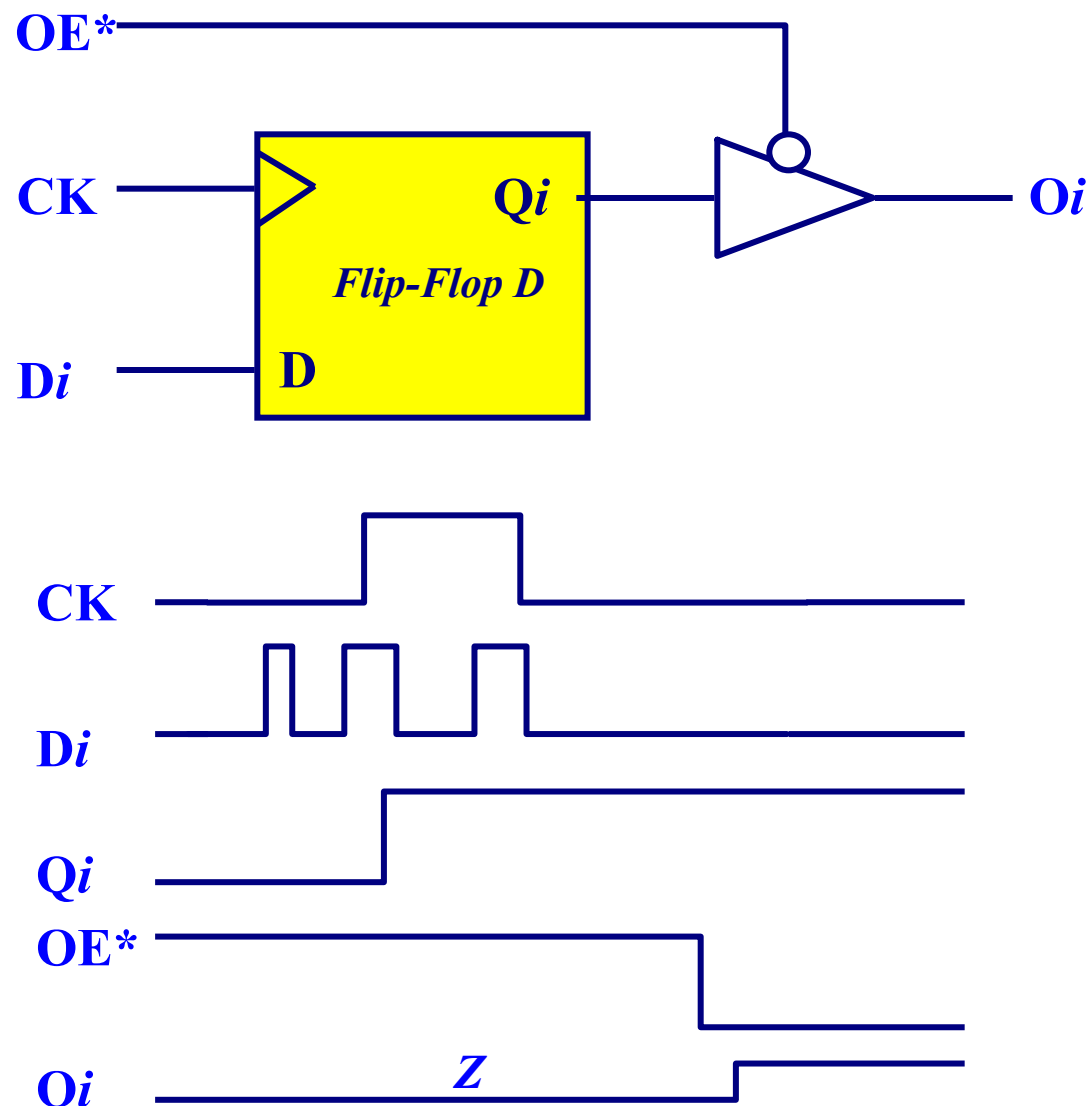


Integrati Notevoli: “374”

74XX374

D0	O0
D1	O1
D2	O2
D3	O3
D4	O4
D5	O5
D6	O6
D7	O7
CK	OE*

Registro *edge-triggered*
con uscite 3-state



SN54LS373, SN54LS374, SN54S373, SN54S374, SN74LS373, SN74LS374, SN74S373, SN74S374

OCTAL D-TYPE TRANSPARENT LATCHES AND EDGE-TRIGGERED FLIP-FLOPS

SDLS165 – OCTOBER 1975 – REVISED MARCH 1988

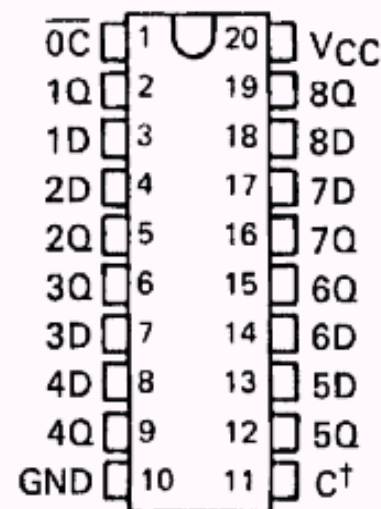
**'LS373, 'S373
FUNCTION TABLE**

OUTPUT ENABLE	ENABLE LATCH	D	OUTPUT
L	H	H	H
L	H	L	L
L	L	X	Q ₀
H	X	X	Z

**'LS374, 'S374
FUNCTION TABLE**

OUTPUT ENABLE	CLOCK	D	OUTPUT
L	↑	H	H
L	↑	L	L
L	L	X	Q ₀
H	X	X	Z

**SN54LS373, SN54LS374, SN54S373,
SN54S374 . . . J OR W PACKAGE
SN74LS373, SN74LS374, SN74S373,
SN74S374 . . . DW OR N PACKAGE
(TOP VIEW)**



description

The eight latches of the 'LS373 and 'S373 are transparent D-type latches meaning that while the enable (C) is high the Q outputs will follow the data (D) inputs. When the enable is taken low the output will be latched at the level of the data that was set up.

The eight flip-flops of the 'LS374 and 'S374 are edge-triggered D-type flip-flops. On the positive transition of the clock, the Q outputs will be set to the logic states that were setup at the D inputs.

Schmitt-trigger buffered inputs at the enable/clock lines of the 'S373 and 'S374 devices, simplify system design as ac and dc noise rejection is improved by typically 400 mV due to the input hysteresis. A buffered output control input can be used to place the eight outputs in either a normal logic state (high or low logic levels) or a high-impedance state. In the high-impedance state the outputs neither load nor drive the bus lines significantly.

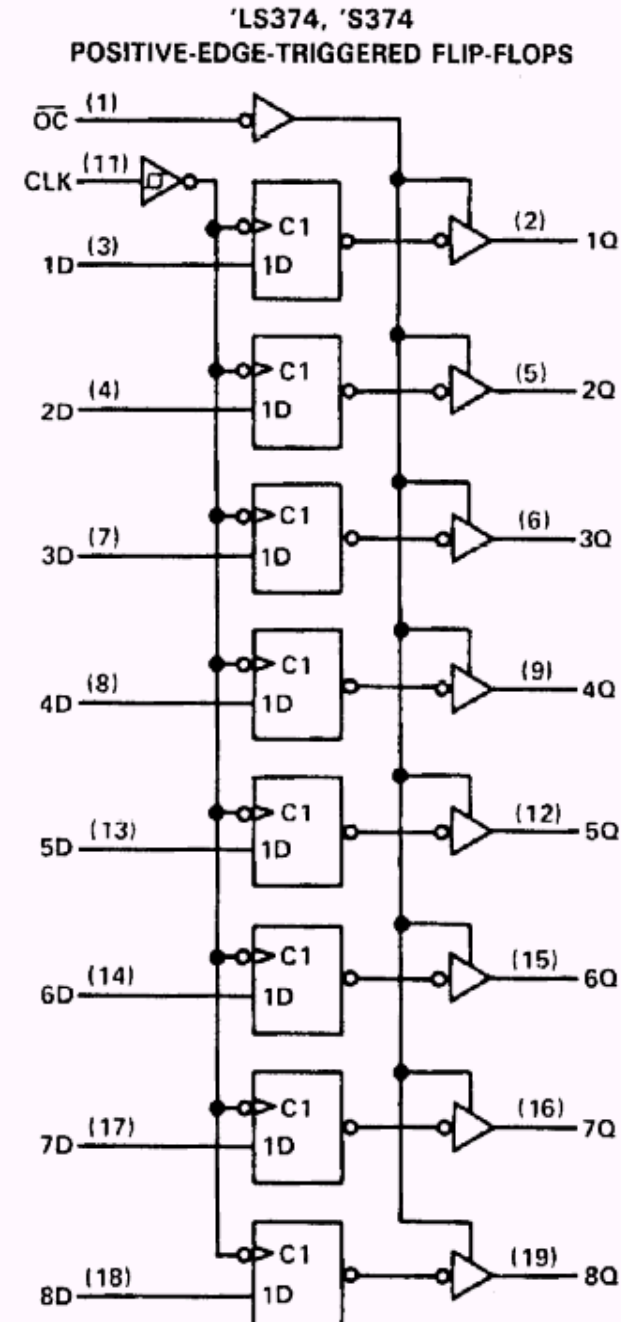
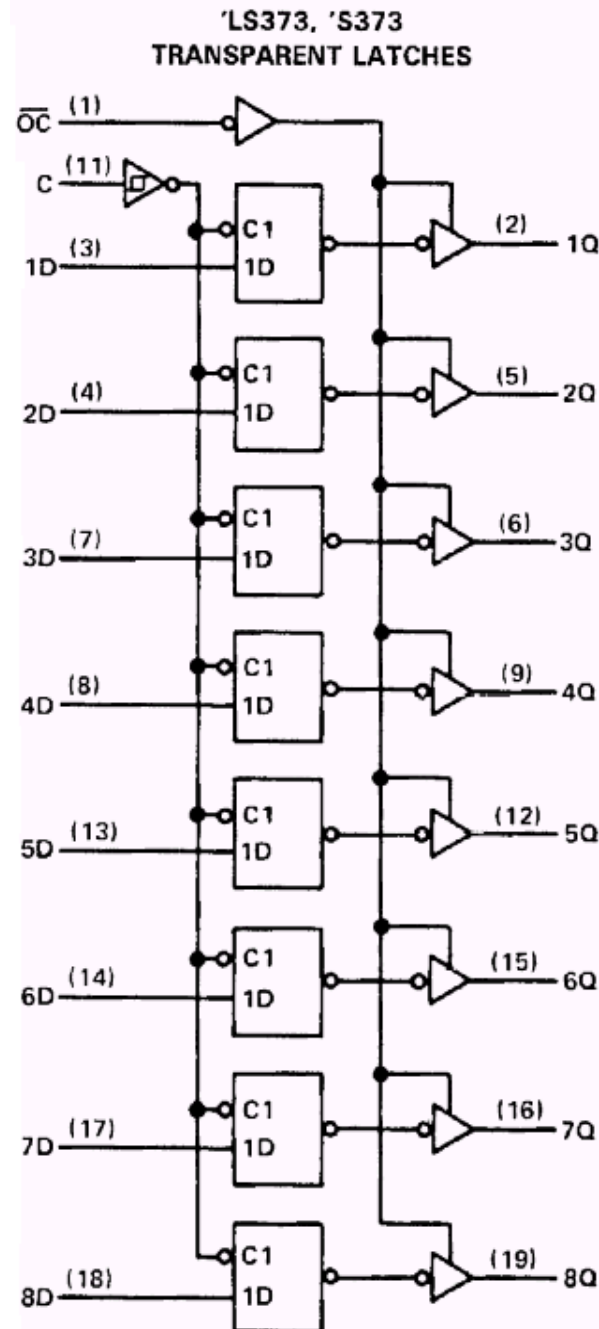
The output control does not affect the internal operation of the latches or flip-flops. That is, the old data can be retained or new data can be entered even while the outputs are off.

SN54LS373, SN54LS374, SN54S373, SN54S374, SN74LS373, SN74LS374, SN74S373, SN74S374

OCTAL D-TYPE TRANSPARENT LATCHES AND EDGE-TRIGGERED FLIP-FLOPS

SDLS165 – OCTOBER 1975 – REVISED MARCH 1988

logic diagrams
(positive logic)



recommended operating conditions

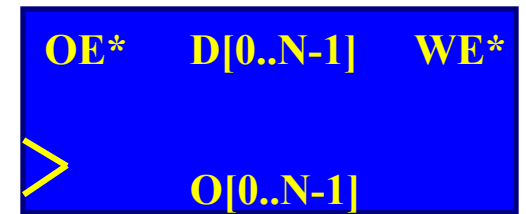
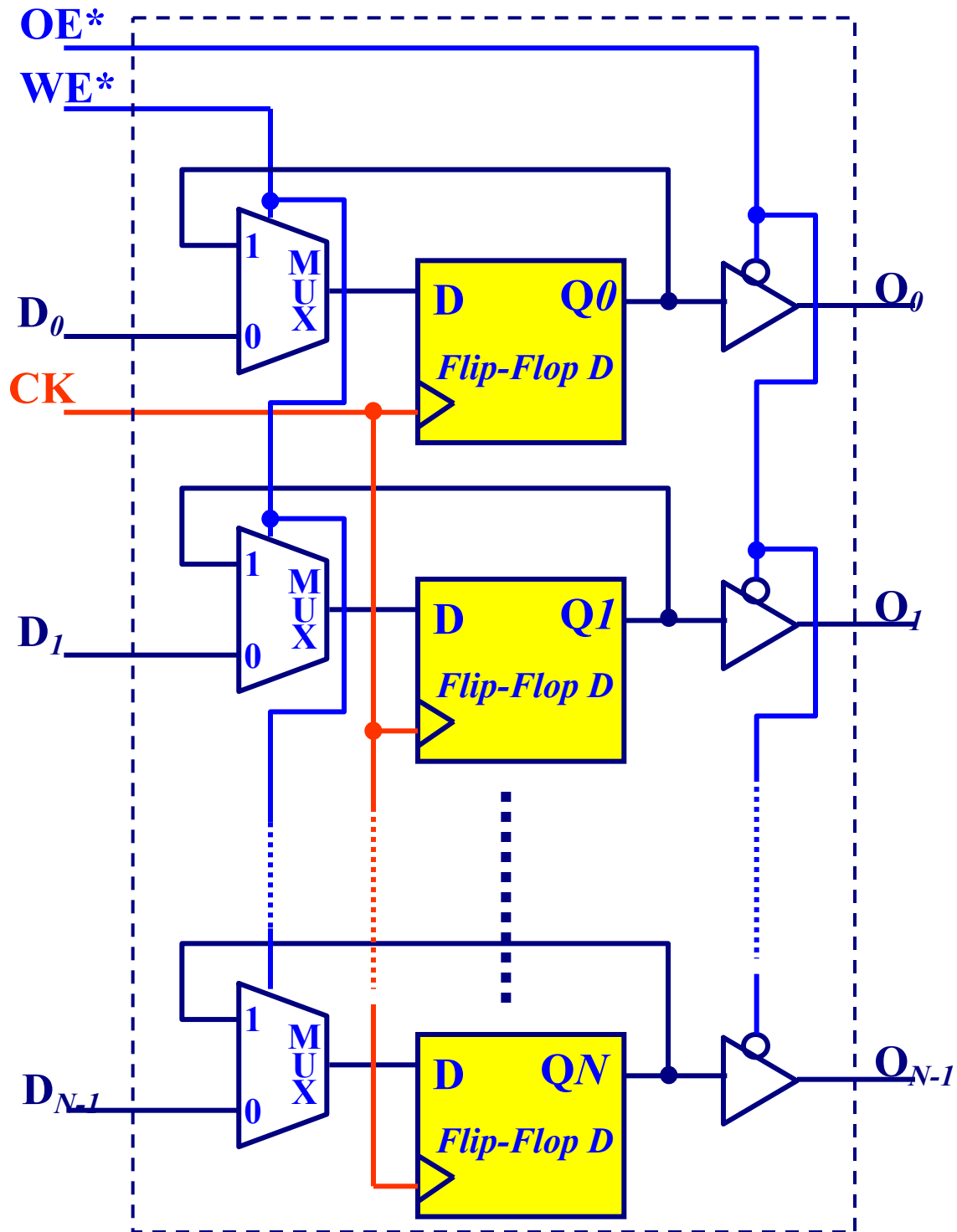
			SN54LS'			SN74LS'			UNIT
			MIN	NOM	MAX	MIN	NOM	MAX	
V _{CC}	Supply voltage		4.5	5	5.5	4.75	5	5.25	V
V _{OH}	High-level output voltage				5.5			5.5	V
I _{OH}	High-level output current				-1			-2.6	mA
I _{OL}	Low-level output current				12			24	mA
t _w	Pulse duration	CLK high	15			15			ns
		CLK low	15			15			
t _{su}	Data setup time	'LS373	5↓			5↓			ns
		'LS374	20↑			20↑			
t _h	Data hold time	'LS373	20↓			20↓			ns
		'LS374↑	5↑			0↑			
T _A	Operating free-air temperature		-55		125	0		70	°C

†The t_h specification applies only for data frequency below 10 MHz. Designs above 10 MHz should use a minimum of 5 ns. (Commercial only)

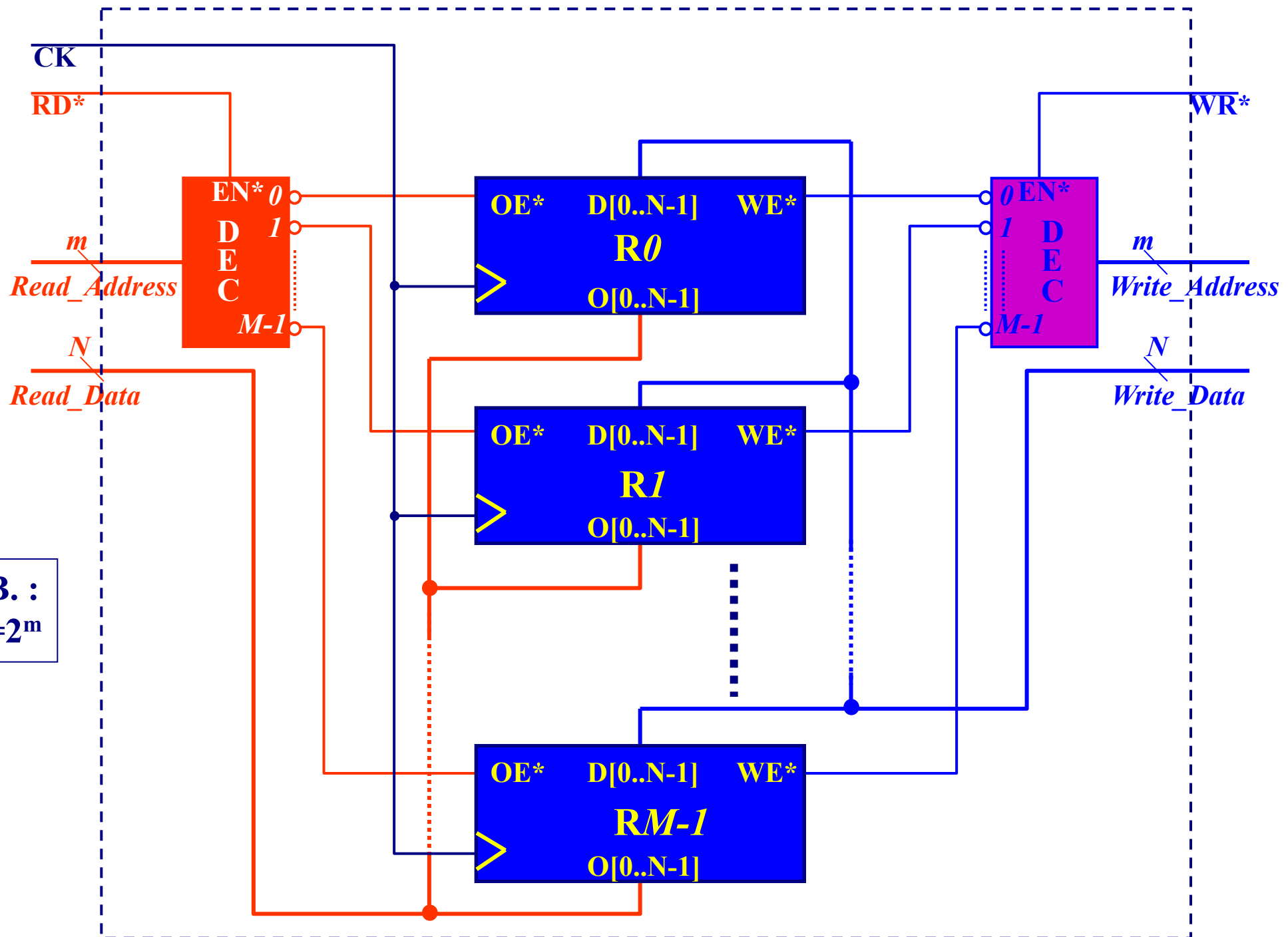
switching characteristics, V_{CC} = 5 V, T_A = 25°C

PARAMETER	FROM (INPUT)	TO (OUTPUT)	TEST CONDITIONS	'LS373			'LS374			UNIT
				MIN	TYP	MAX	MIN	TYP	MAX	
f _{max}			C _L = 45 pF, R _L = 667 Ω See Notes 2 and 3				35	50		MHz
t _{PLH}	Data	Any Q			12	18				ns
t _{PHL}					12	18				
t _{PLH}	Clock or enable	Any Q			20	30		15	28	ns
t _{PHL}					18	30		19	28	
t _{pZH}	Output Control	Any Q			15	28		20	26	ns
t _{pZL}					25	36		21	28	
t _{PHZ}	Output Control	Any Q		C _L = 5 pF, R _L = 667 Ω See Note 3		15	25		15	28
t _{PLZ}	Output Control	Any Q			12	20		12	20	ns

Registro Edge-Triggered con WE*



Register File (1 read-port, 1 write-port)



Mapping di dispositivi da 8 bit in sistemi con bus dati da 8 bit

- Consideriamo dispositivi con porta dati a 8 bit
- Imponiamo (temporaneamente) l'ulteriore condizione che il parallelismo del bus dati sia a 8 bit
- In queste ipotesi l'assegnamento a un dispositivo di una finestra di indirizzi in uno spazio di indirizzamento avverrà in generale nel rispetto delle due seguenti ulteriori condizioni restrittive:
 - *la dimensione della finestra di indirizzi associata a un dispositivo è una potenza di due*
 - *la finestra è composta da indirizzi contigui*

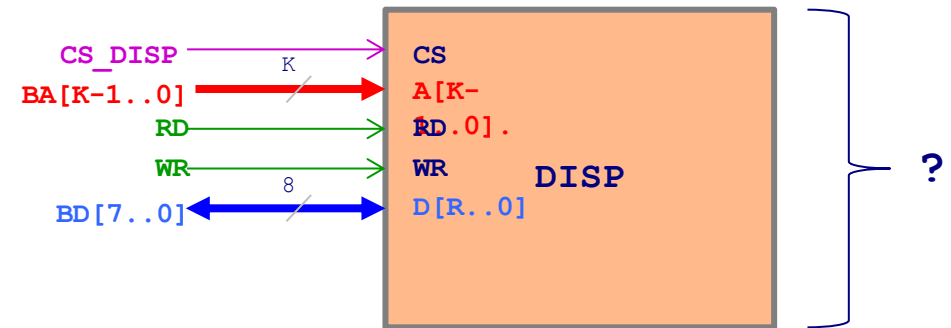
Dimensione della finestra occupata da un dispositivo - esempi

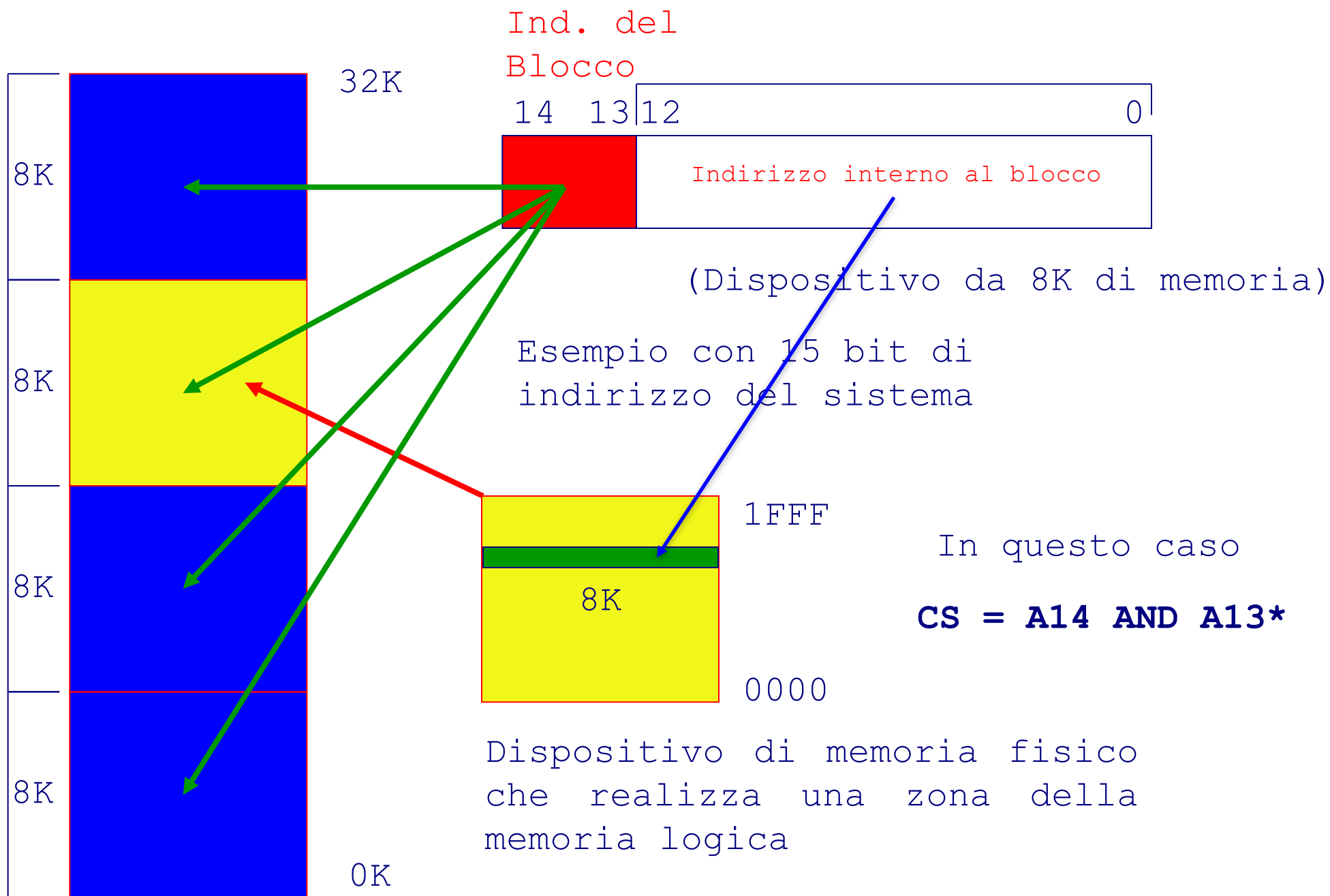
- Un dispositivo accessibile attraverso il bus occupa in generale $n = 2^K$ posizioni nello spazio di indirizzamento
- n rappresenta il numero di *oggetti* a 8 bit indirizzabili all'interno del dispositivo (es. numero di celle di memoria nelle RAM ed EPROM)
- K (numero di bit di indirizzo *interni* al dispositivo) è fortemente variabile al variare del dispositivo:
 - In generale nei dispositivi di input/output (i.e., le interfacce) K è piccolo (e.g., 2)
 - in generale nei dispositivi di memoria K è grande (e.g., per una RAM da 128 KB si ha $K = 17$)

Caratteristiche ai morsetti di un dispositivo indirizzabile su una finestra di $n = 2^K$ byte

Qualunque dispositivo da 8 bit con all'interno $n = 2^k$ elementi indirizzabili separatamente ha al suo interno un decoder (II livello) di K variabili con ingresso di enable che seleziona i singoli oggetti indirizzabili

- Read (**RD**), detto anche Output Enable (OE) è il comando di lettura. Quando RD e CS sono attivi, il dispositivo espone il su **BD[7..0]** il contenuto della cella indirizzata
- Write (**WR**), è il comando di scrittura. Quando CS asserito sul fronte di discesa di WR è campionato il dato presente su **BD[7..0]**





Mapping allineato di dispositivi da 8 bit in sistemi con bus dati da 8 bit

Si consideri un dispositivo D di $n=2^K$ byte indirizzabili

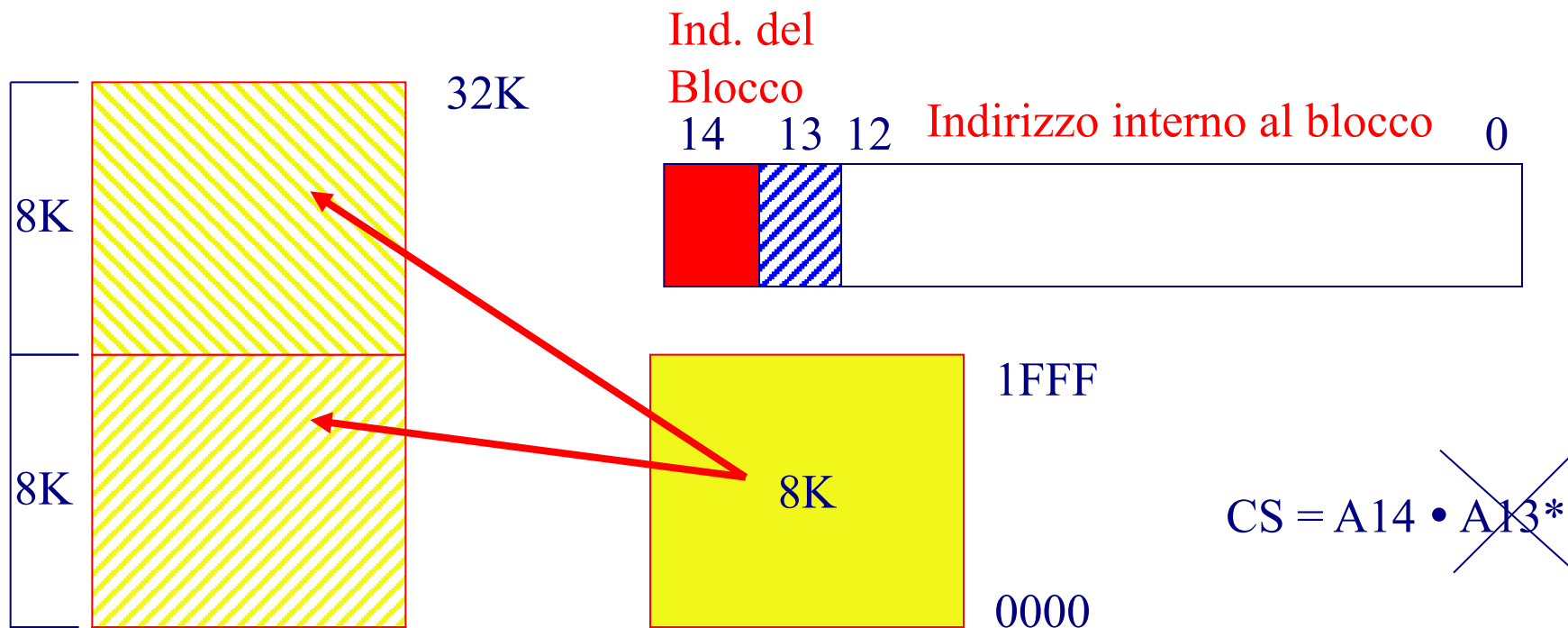
- Si dice che D è mappato all'indirizzo A se gli indirizzi dei byte di D sono compresi tra A e $A+(n-1)$, cioè se A è l'indirizzo più basso tra tutti gli indirizzi associati a D
- Si dice che D è allineato se A è un multiplo di n (numero di byte interni al dispositivo), cioè se:
$$(\text{indirizzo più basso di D}) \bmod n = 0 \quad (\text{condizione di allineamento})$$
- Se D è allineato allora i k bit meno significativi di A sono uguali a zero

Esempi:

- Un dispositivo da due byte è allineato se è mappato a un indirizzo pari
- Una dispositivo da 8 byte è allineato se è mappato a un indirizzo il cui valore codificato in binario termina con 3 zeri
- Un dispositivo da 16 byte è allineato se il suo indirizzo iniziale in codice esadecimale ha la cifra meno significativa uguale a zero
- Un dispositivo da 64 KB è allineato se il suo indirizzo in codice esadecimale ha le quattro cifre meno significative uguali a zero

Decodifica degli indirizzi in caso di mapping allineato

- Consideriamo uno spazio di indirizzamento di 1 MB in cui sia mappato un dispositivo di 2^K byte
- Per individuare una cella di indirizzo $A_i = \alpha \# i$ possiamo decodificare tutti i 20 bit che compongono A_i
- Questa decodifica è effettuata ricorrendo alla struttura dei decoder ad albero, con albero di due livelli:
 - Il I livello è usato per decodificare α (che identifica la posizione in cui il chip è mappato); per decodificare α dobbiamo decodificare 20-K variabili
 - il II livello viene utilizzato per decodificare i (che identifica il byte all'interno del chip, serve un decoder di k variabili)
- Il decoder di II livello si trova all'interno del chip mentre la decodifica di α è a carico del progettista del sistema che può utilizzare un decoder di 20-k variabili con cui si decodifica α
- La decodifica è completa se si utilizzano tutti i 20-K bit per decodificare α , semplificata se si utilizza solo un sottoinsieme (minimo) dei 20-K bit



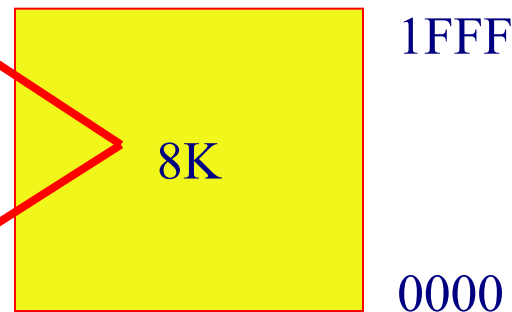
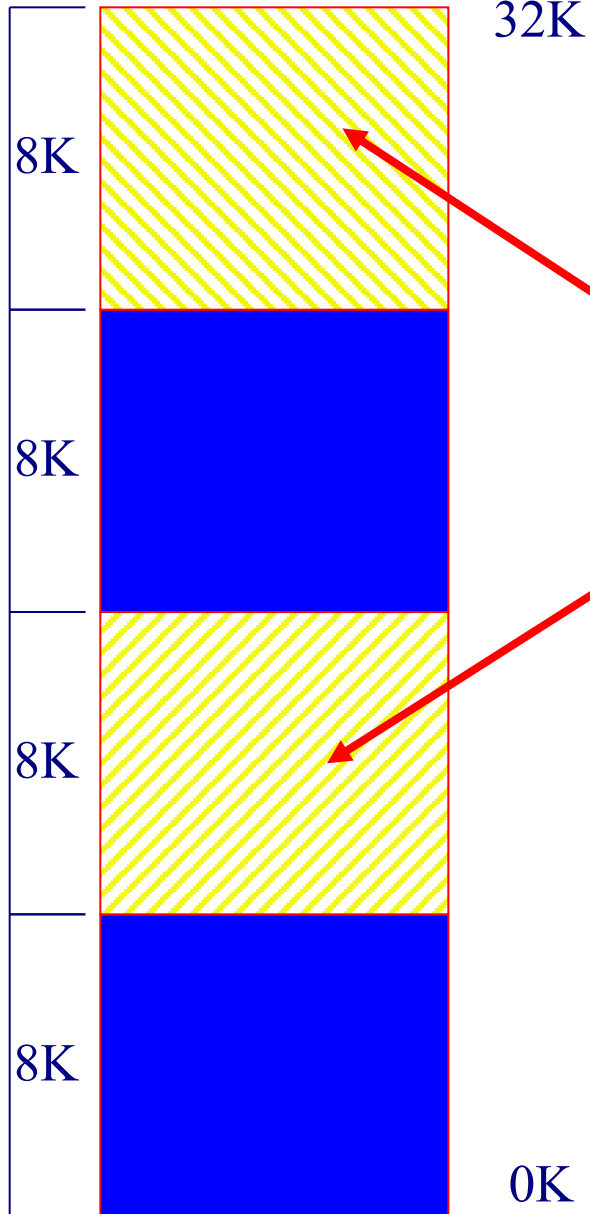
Il modulo di memoria fisico è di fatto attivato (mappato) in due differenti zone della memoria logica

Se gli indirizzi usati da un programma sono quelli che vanno da 16K a 24K (overo da 4000H a 5FFFH) e quelli da 24K a 32K (da 6000H a 7FFF) non sono usati allora è possibile la decodifica incompleta o parziale in quanto la zona 24K-32K non viene mai indirizzata. Espressione CS più semplice

Decodifica parziale 2/2

Ind. del
Blocco

14 13 12 Indirizzo interno al blocco 0



$$\text{CS} = \cancel{A_{14}} \cdot A_{13}$$

Il modulo di memoria fisico è
attivato (mappato) in due differenti
zone della memoria logica **non**
consecutive

Spazio di memoria

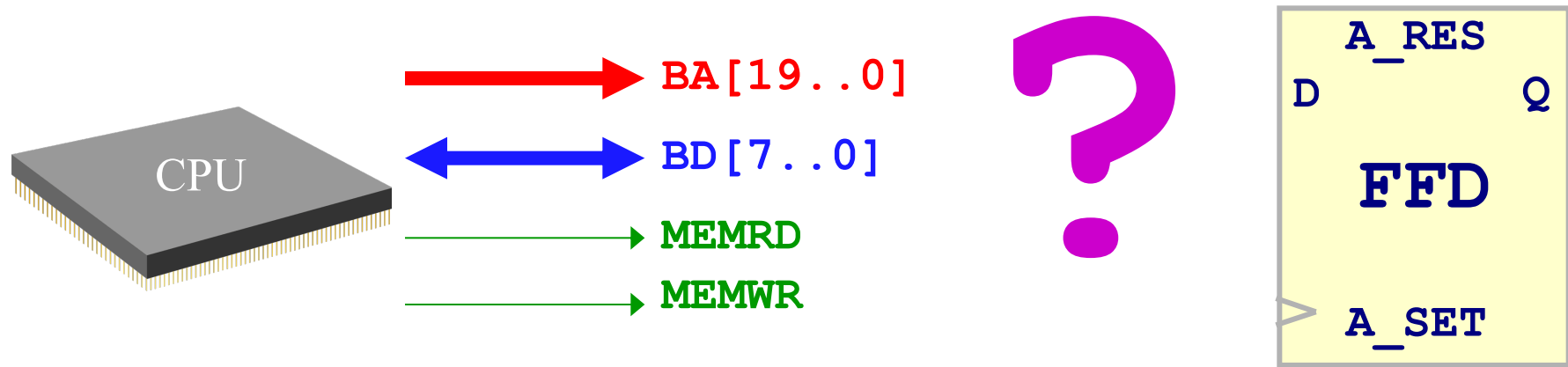
Esercizio

Si consideri un sistema con bus indirizzi a 16 bit e bus dati a 8 bit. Scrivere le espressioni di decodifica completa e semplificata (quella da usare all'esame) nei seguenti casi:

- 1) Dispositivo di memoria da 16 KB mappato a 8000h
- 2) Dispositivo di memoria da 8 KB mappato a 0000h
- 3) Entrambi i dispositivi precedenti

Se c'è un solo dispositivo (casi 1 e 2), il CS è molto particolare...

Mapping, read, write e set/reset di un FFD



- Il FFD è un elementare dispositivo di memoria
- Con una CPU, come possiamo:
 - **scrivere** nel FFD
 - **leggere** nel FFD
 - **settare** o **resettare** in attraverso i comandi asincroni il FFD

Consideriamo il caso di una **CPU con bus dati a 8 bit** con **20 bit di indirizzo**. **64 K di RAM** agli indirizzi alti e **64 K di EPROM** agli indirizzi bassi

Nelle pagine seguenti assumiamo che i comandi del FFD siano mappati nei seguenti indirizzi:

CS_READ_FFD	->	80003h
CS_WRITE_FFD	->	80002h
CS_A_RES_FFD	->	80001h
CS_A_SET_FFD	->	80000h

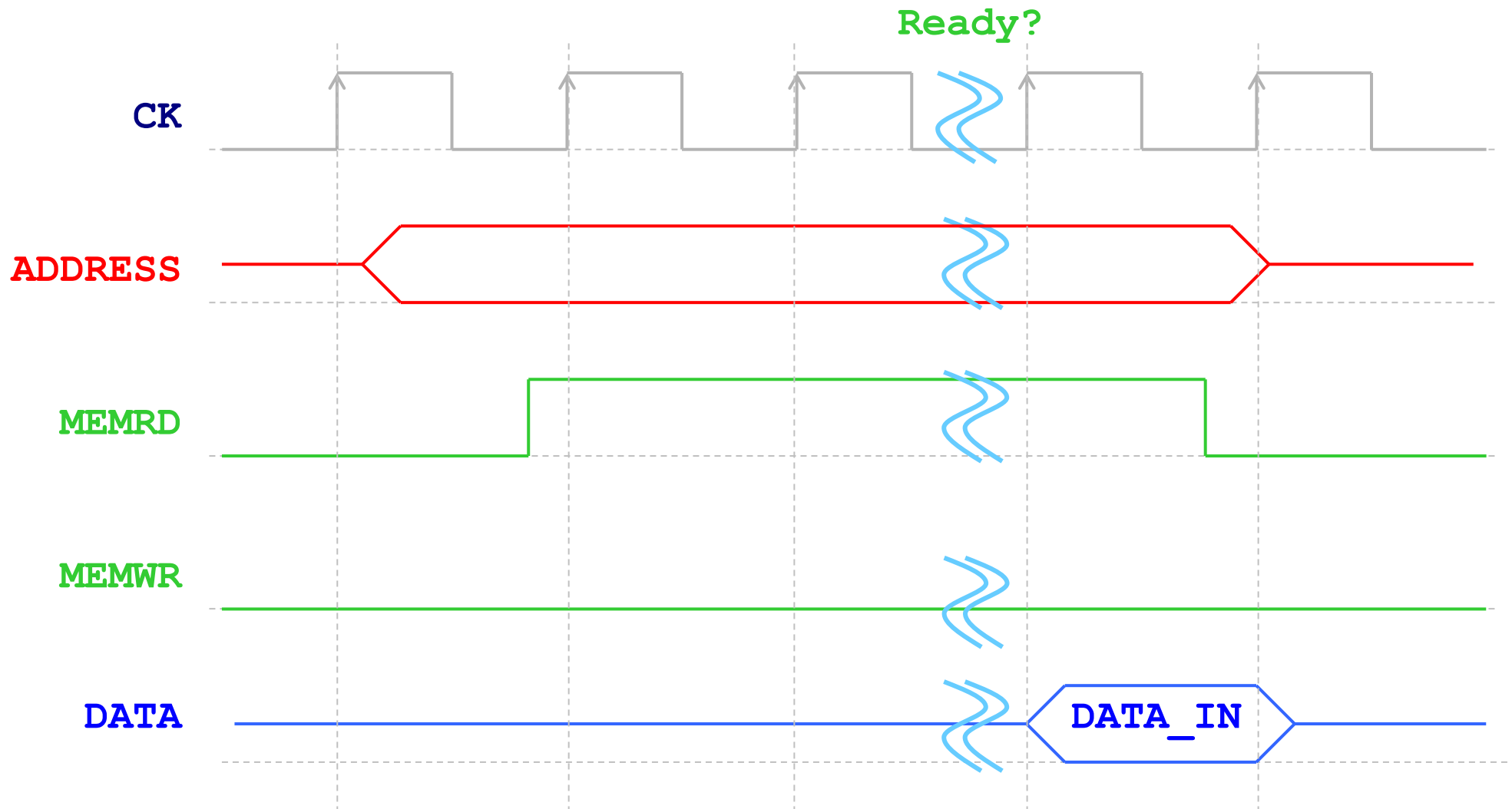
Assumiamo inoltre di utilizzare il segnale **BD0** del bus dati per leggere e scrivere il singolo bit di dato.

Ovviamente sarebbe possibile utilizzare altri indirizzi non appartenenti alle memorie e anche altri segnali del bus dati (anche diversi per letture e scritture).

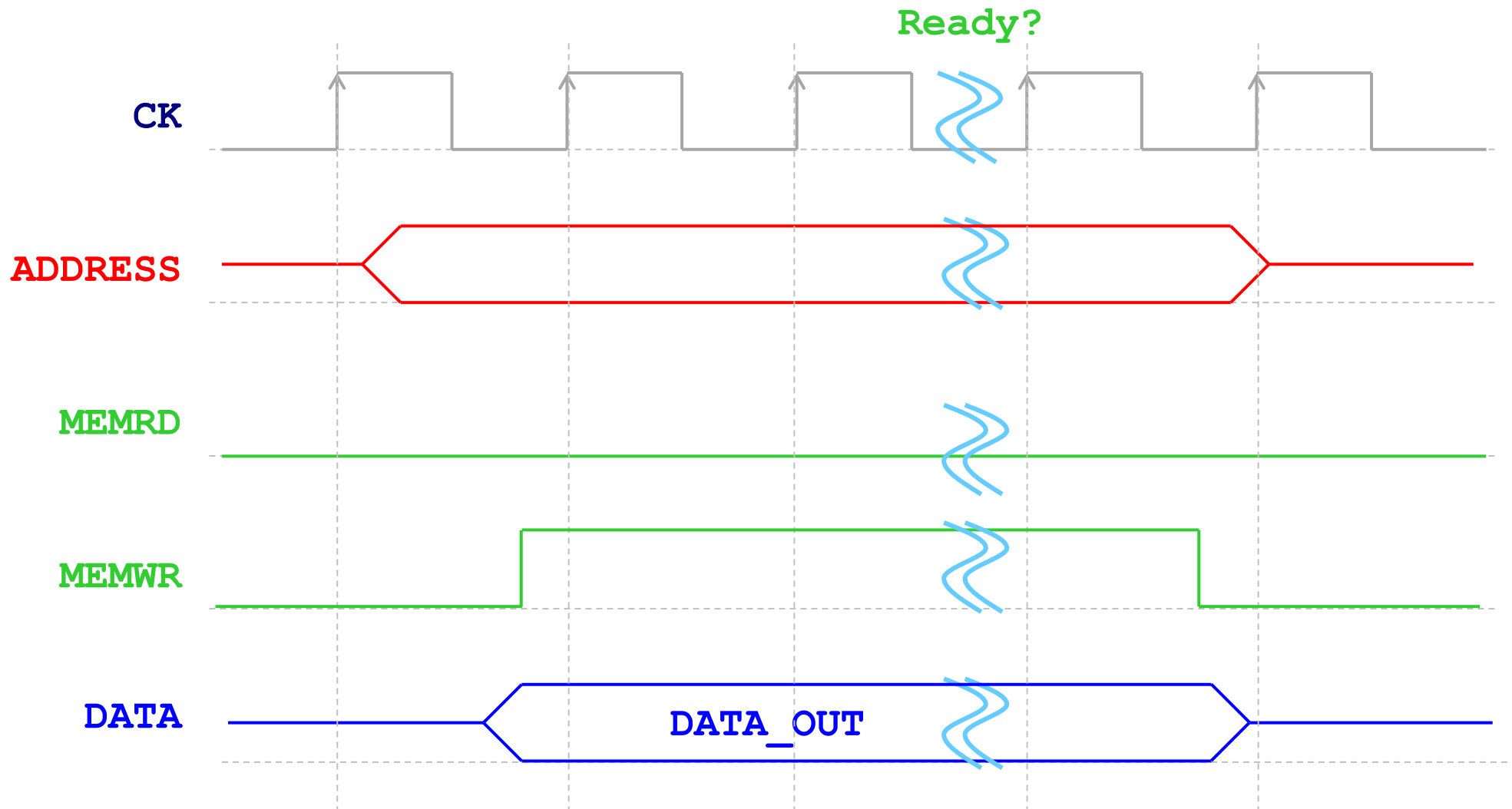
Se il testo dell'esame non specifica quali indirizzi usare la scelta è lasciata allo studente.

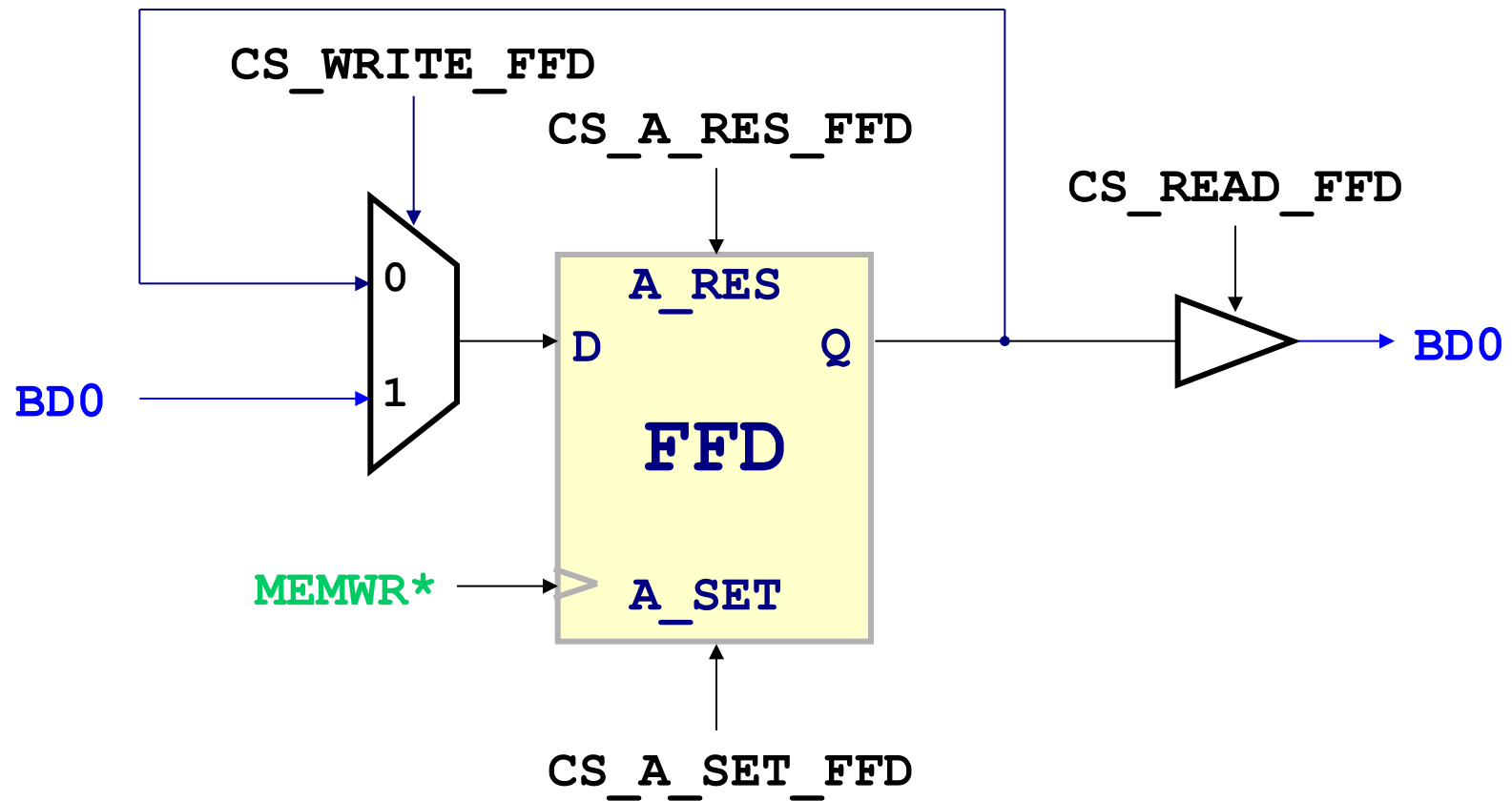
Spesso, la scelta degli indirizzi semplifica/complica i segnali di decodifica.

Esempio di ciclo di lettura



Esempio di ciclo di scrittura

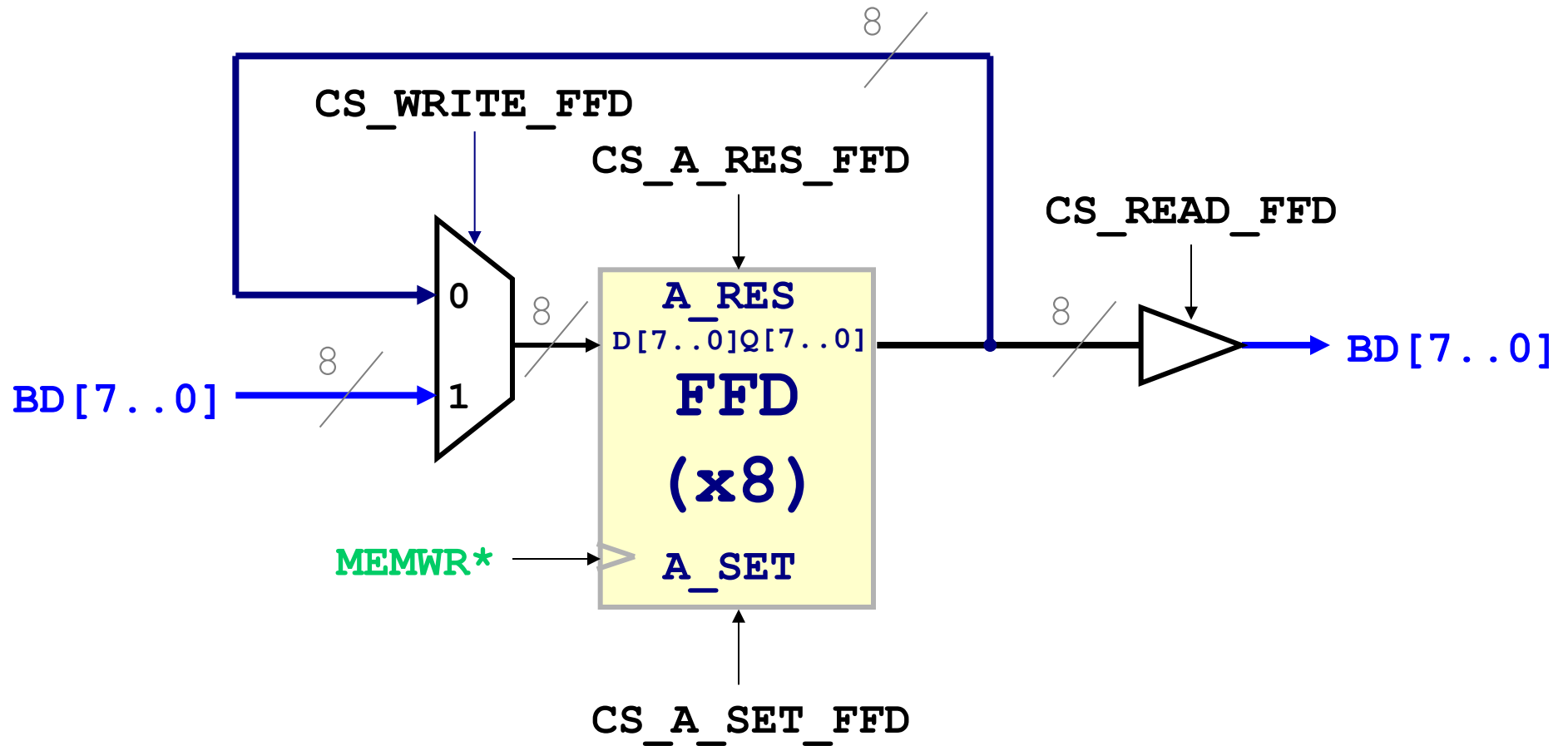




CS_RAM_H	=	BA19 · BA18 · BA15	
CS_RAM_L	=	BA19 · BA18 · BA15*	
CS_READ_FFD	=	BA19 · BA18* · BA1 · BA0 · MEMRD	(ist. lettura)
CS_WRITE_FFD	=	BA19 · BA18* · BA1 · BA0*	(ist. scrittura)
CS_A_RES_FFD	=	BA19 · BA18* · BA1* · BA0 · MEMWR	(ist. scrittura)
CS_A_SET_FFD	=	BA19 · BA18* · BA1* · BA0* · MEMWR	(ist. scrittura)
CS_EPROM	=	BA19*	

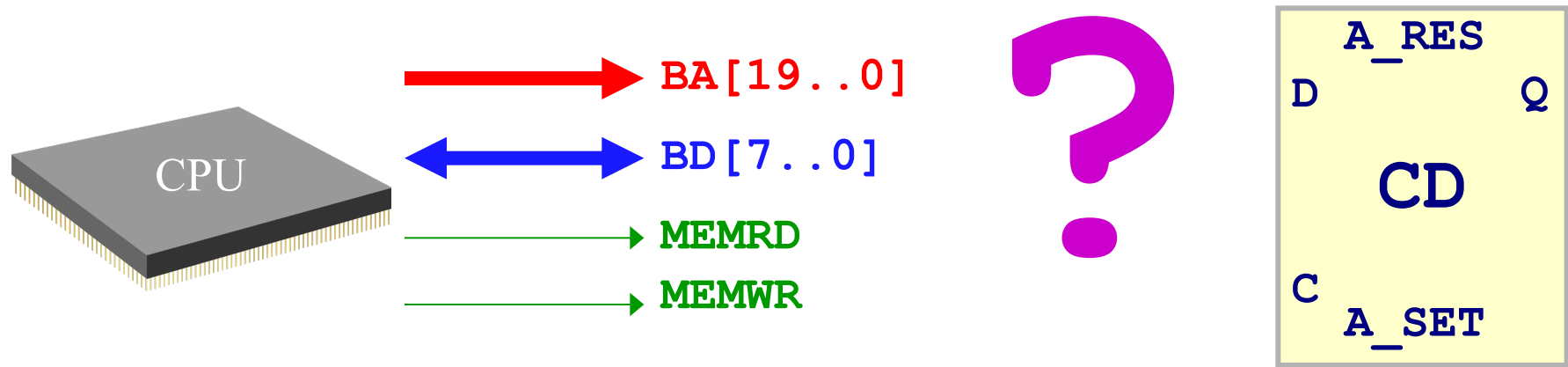
Vedremo che istruzioni di lettura e scrittura sono (risp.)
load byte (LB) e *store byte* (SB).

Estensione a 8 bit



Stessi CS della pagina precedente, cambia solo il numero di bit di dato trasferiti.

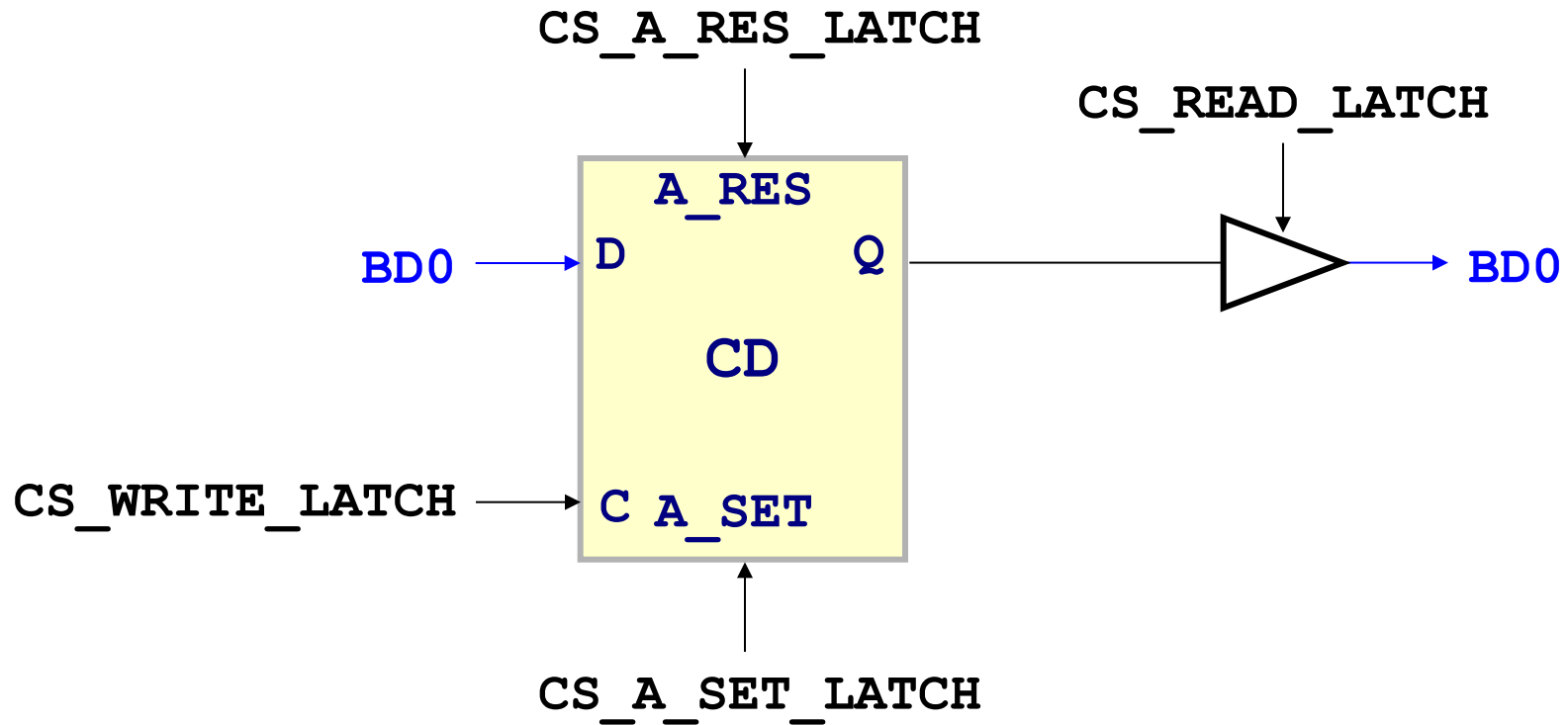
Mapping, read, write e set/reset di un latch



- Anche il latch CD è un elementare dispositivo di memoria
- Con una CPU, come possiamo:
 - **scrivere** nel latch
 - **leggere** nel latch
 - **settare** o **resettare** in modo asincrono il latch

Consideriamo il caso di una **CPU con bus dati a 8 bit** con **20 bit di indirizzo**. **64 K di RAM** agli indirizzi alti e **64 K di EPROM** agli indirizzi bassi

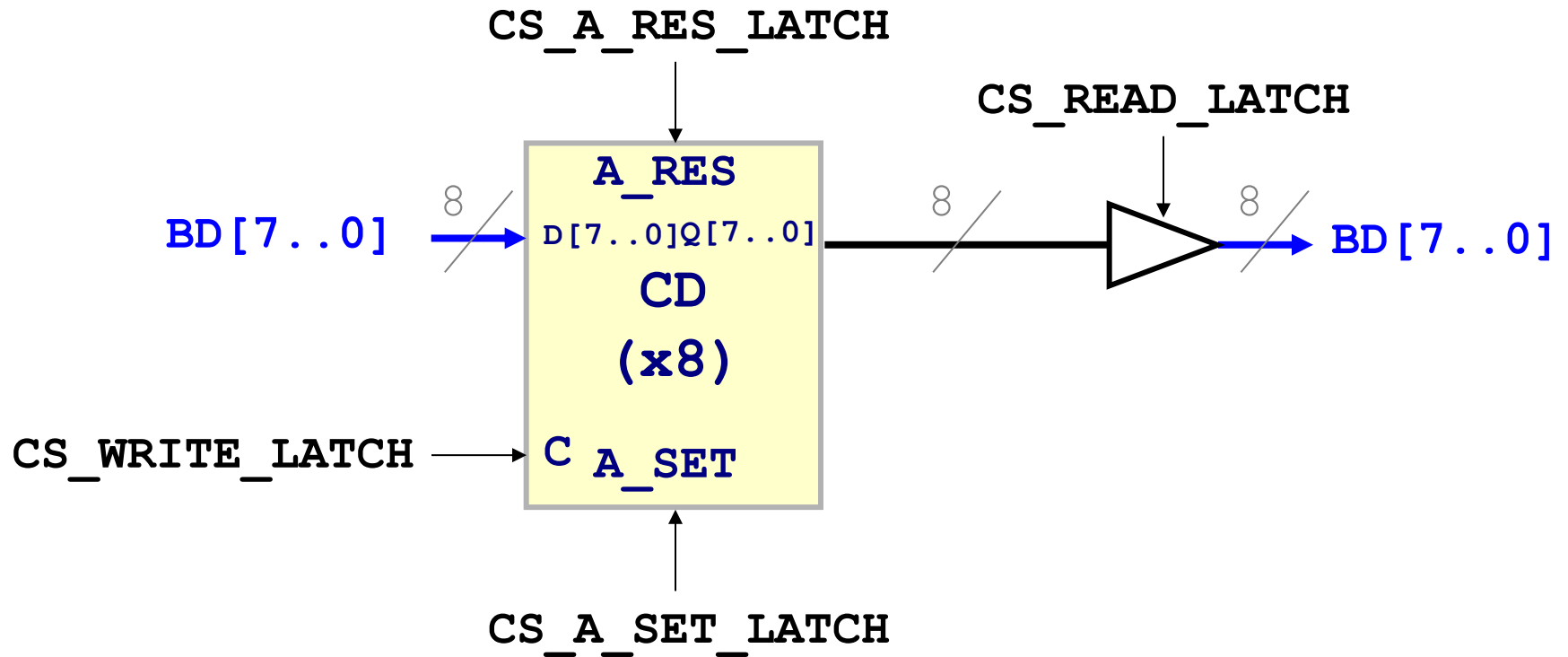
Mappiamo i quattro comandi del latch agli stessi indirizzi usati per il FFD.



$CS_RAM_H = BA19 \cdot BA18 \cdot BA15$
 $CS_RAM_L = BA19 \cdot BA18 \cdot BA15^*$
 $CS_READ_LATCH = BA19 \cdot BA18^* \cdot BA1 \cdot BA0 \cdot MEMRD$ (ist. lettura)
 $CS_WRITE_LATCH = BA19 \cdot BA18^* \cdot BA1 \cdot BA0^* \cdot MEMWR$ (ist. scrittura)
 $CS_A_RES_LATCH = BA19 \cdot BA18^* \cdot BA1^* \cdot BA0 \cdot MEMWR$ (ist. scrittura)
 $CS_A_SET_LATCH = BA19 \cdot BA18^* \cdot BA1^* \cdot BA0^* \cdot MEMWR$ (ist. scrittura)
 $CS_EPROM = BA19^*$

Vedremo che istruzioni di lettura e scrittura sono (risp.)
load byte (LB) e *store byte* (SB).

Estensione a 8 bit



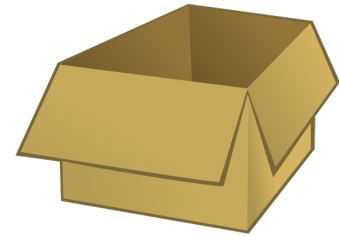
Stessi CS della pagina precedente, cambia solo il numero di bit di dato trasferiti.

Incrementare il parallelismo dei dati

- Abbiamo considerato fino a ora sistemi con un parallelismo (bus dati) a 8 bit
- Ogni trasferimento richiede un ciclo di bus
- N elementi (byte) $\rightarrow N$ cicli di bus
- Sappiamo che le memorie (e non solo) sono lente (vs CPU)

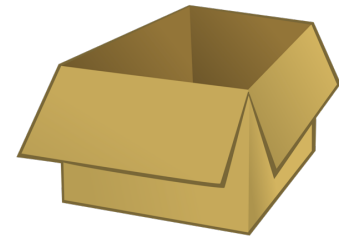


i)



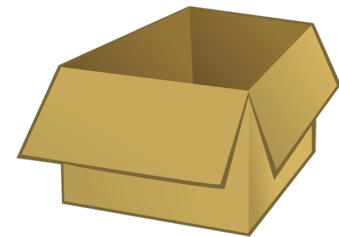
1

ii)



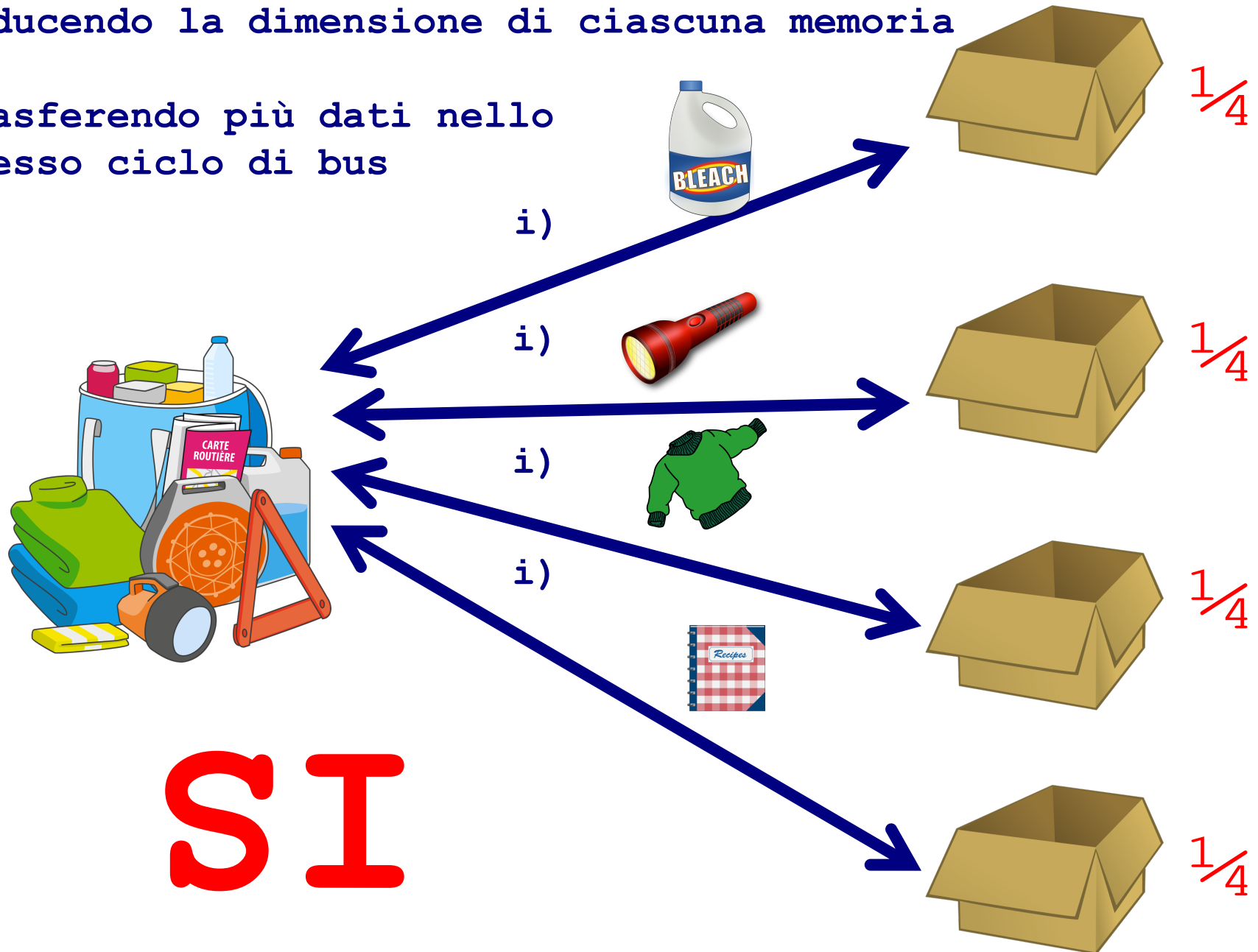
1

iii)

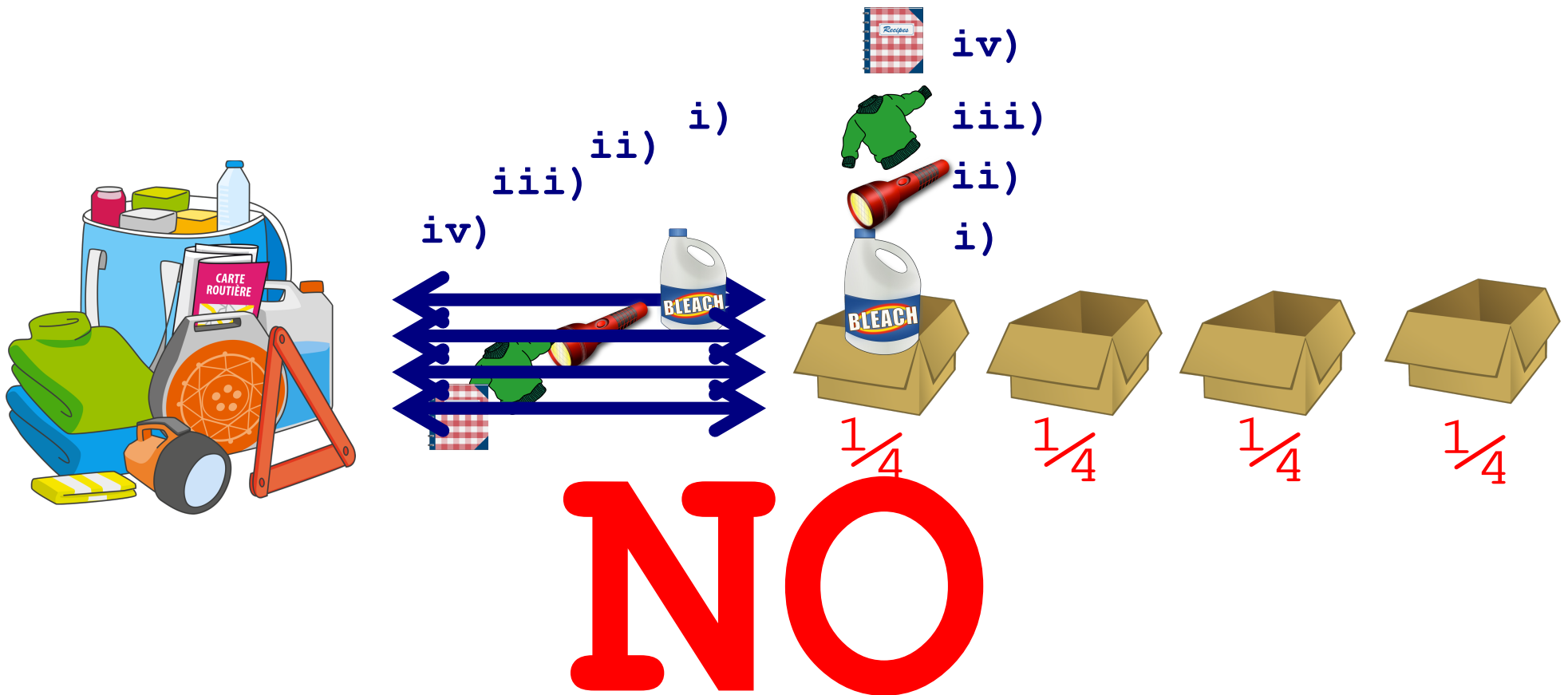


1

- Possiamo fare meglio?
- Sì, aumentando il parallelismo dei dati
- Riducendo la dimensione di ciascuna memoria
- Trasferendo più dati nello stesso ciclo di bus

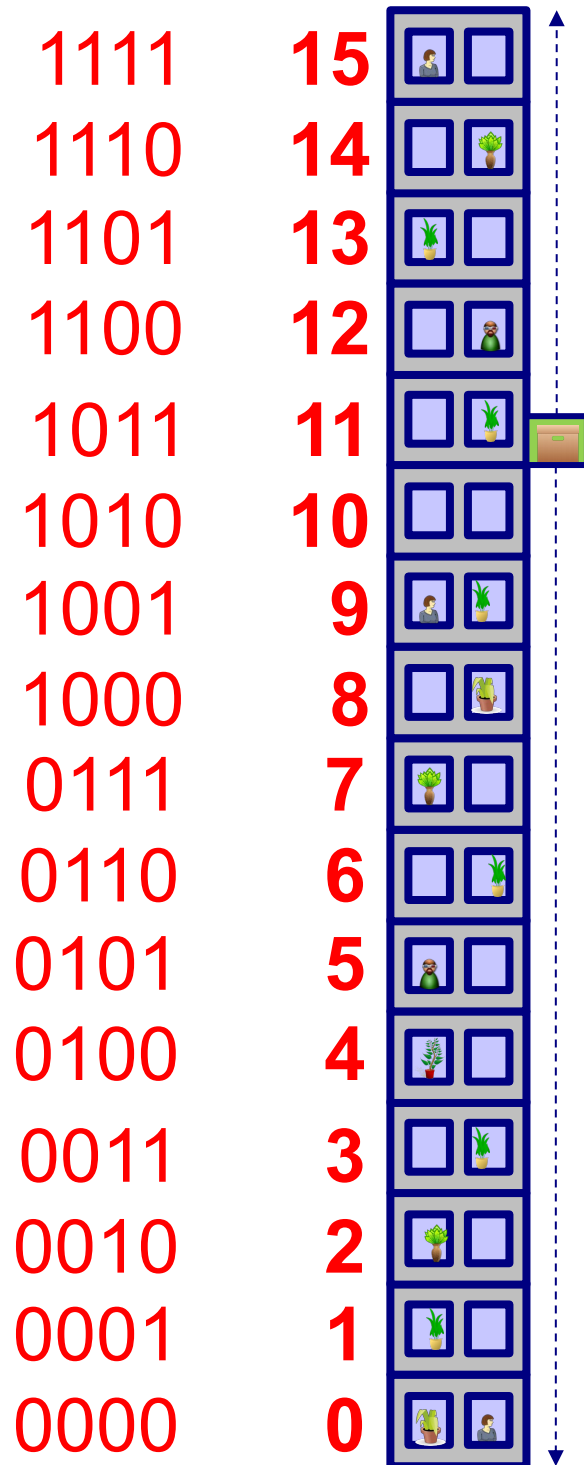


- Cosa NON fare?
- Trasferire gli elementi sequenzialmente in memorie più piccole
- Elementi contigui vanno su memorie diverse

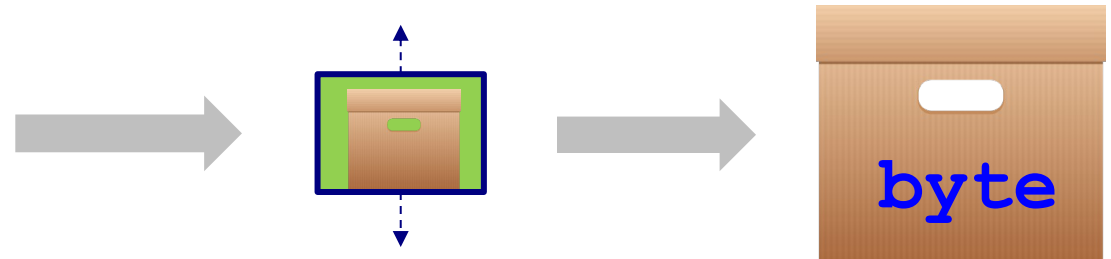


il parallelismo di ciascuna memoria è sempre 8 bit!

Altro esempio

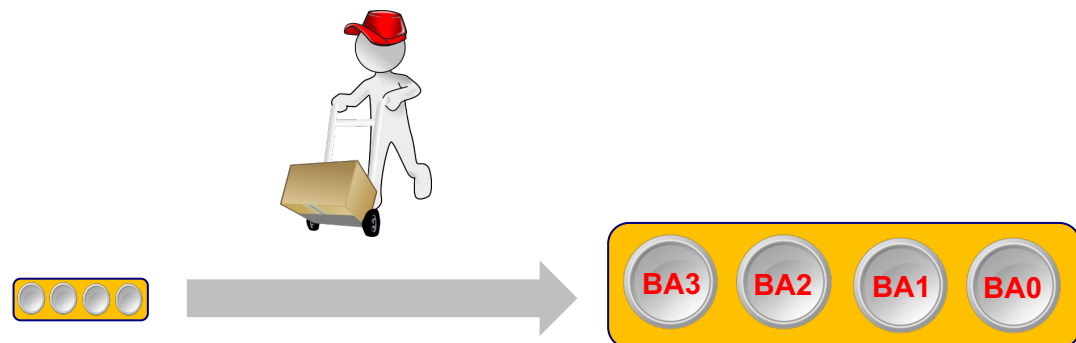


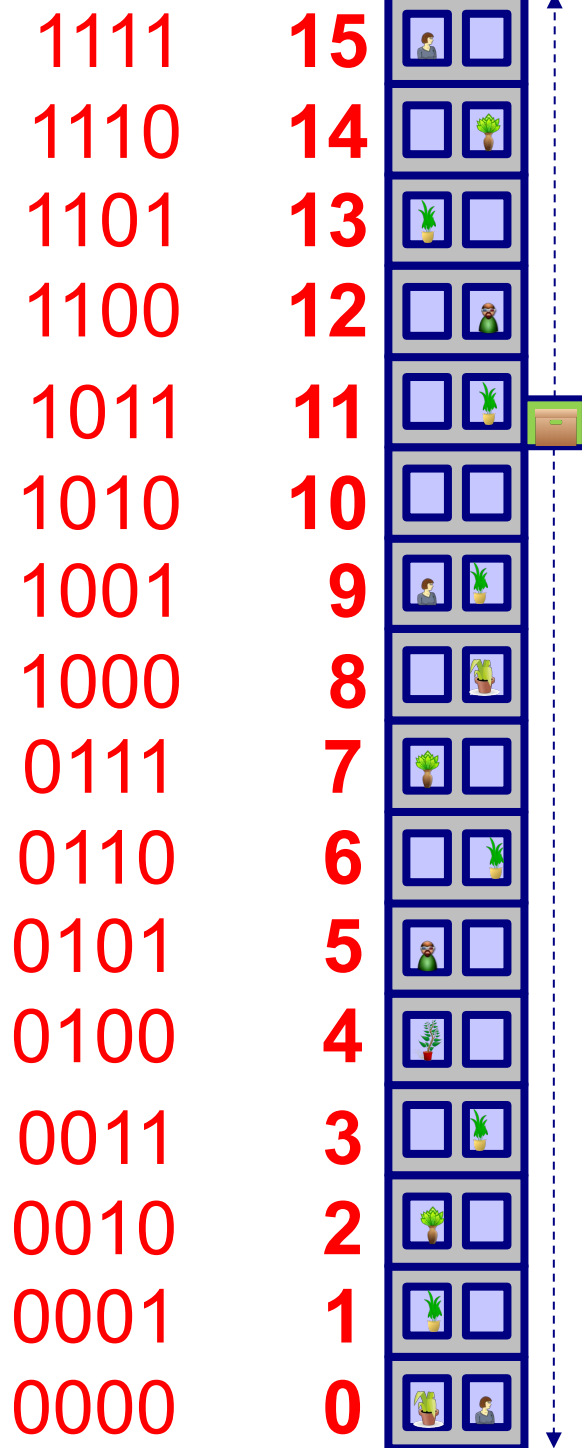
Edificio con un appartamento per ogni piano e un **montacarichi** che può inviare **una sola scatola/byte per volta**



Per selezionare il piano, sono presenti **4 pulsanti** (i.e., 4 bit):
BA3, BA2, BA1, BA0

I 4 pulsanti consentono di selezionare uno dei 16 piani



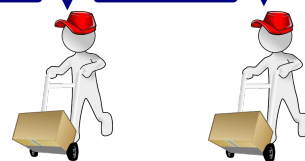
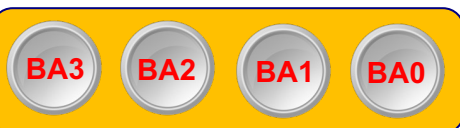
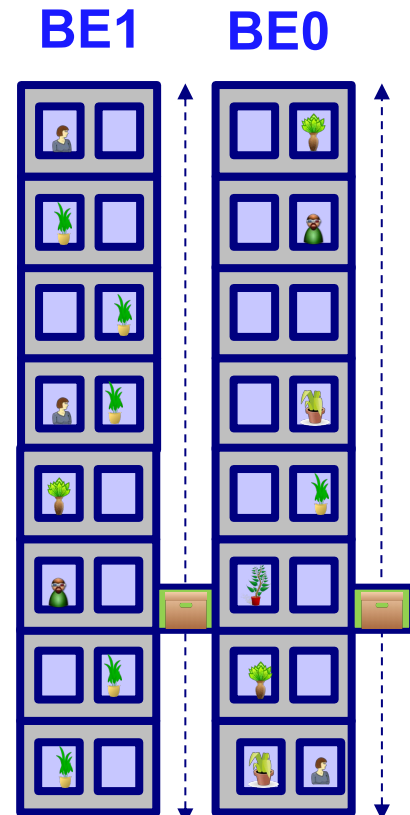


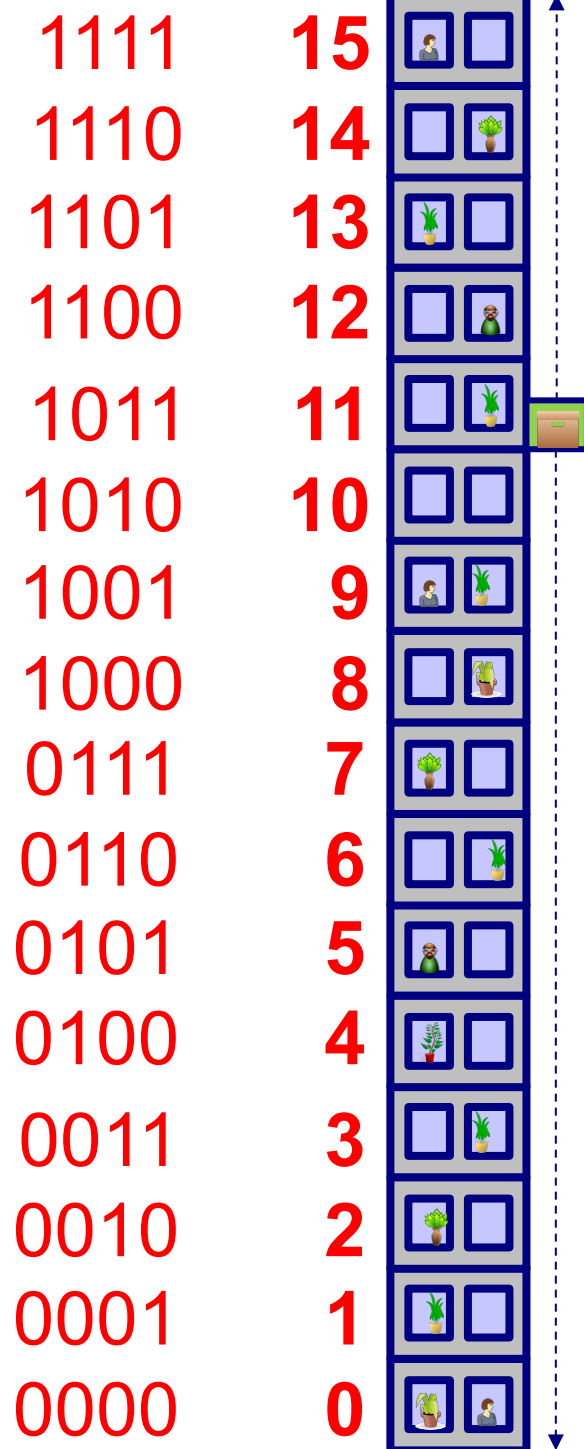
Un'alternativa (in basso a DX): due edifici distinti con un appartamento per ogni piano e un montacarichi per ogni edificio

Una sola pulsantiera con la quale è possibile inviare **tutti i montacarichi allo stesso piano**

Qual è il vantaggio?
 I segnali **BE_i** cosa indicano?
 Il segnale **BA₀**?

111	7
110	6
101	5
100	4
011	3
010	2
001	1
000	0



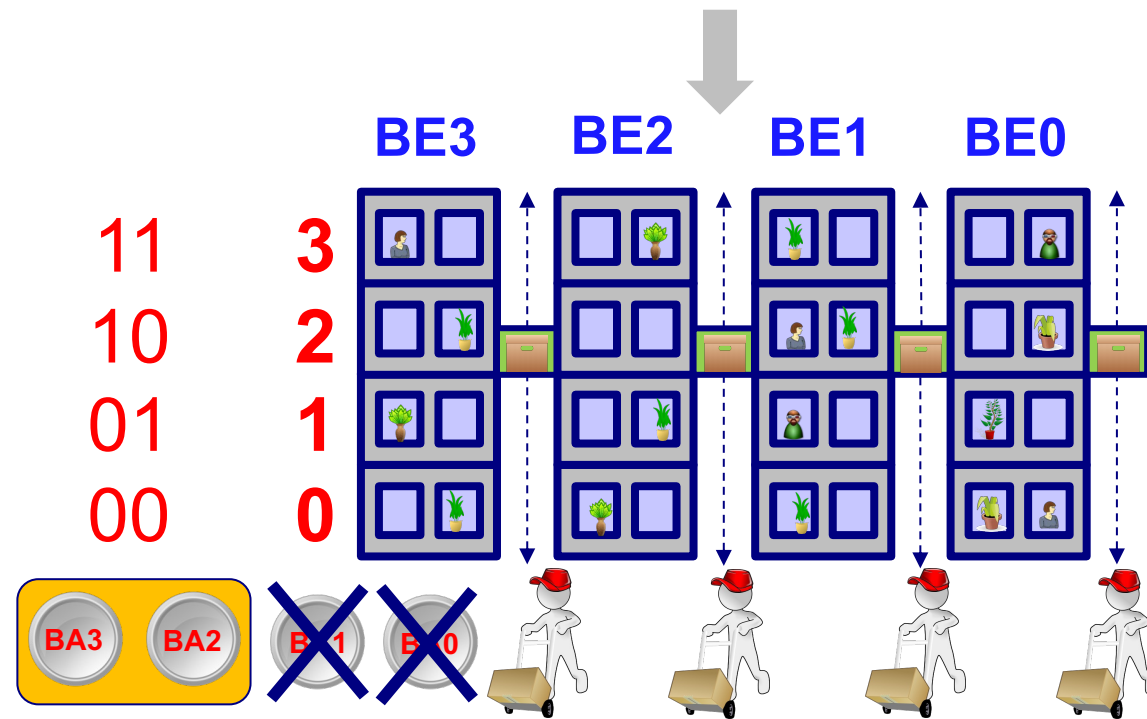


Un'altra soluzione (in basso a DX);
quattro edifici distinti con un
appartamento per ogni piano e un
montacarichi per ogni edificio

**Una sola pulsantiera e tutti i
montacarichi possono essere inviati
unicamente allo stesso piano**

Qual è il vantaggio rispetto alla
soluzione precedente?

I segnali **BA1** e **BA0**?



Negli esempi precedenti, le pulsantiere avrebbero i seguenti pulsanti/segnali:

Caso 1 edificio:



Caso 2 edifici:

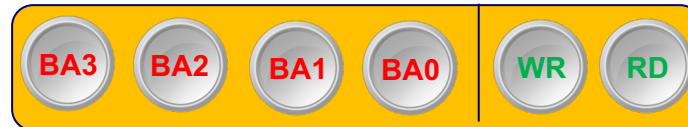


Caso 4 edifici:



Nel caso si volessero prelevare e inviare scatole/dati:

Caso 1 edificio:



Caso 2 edifici:

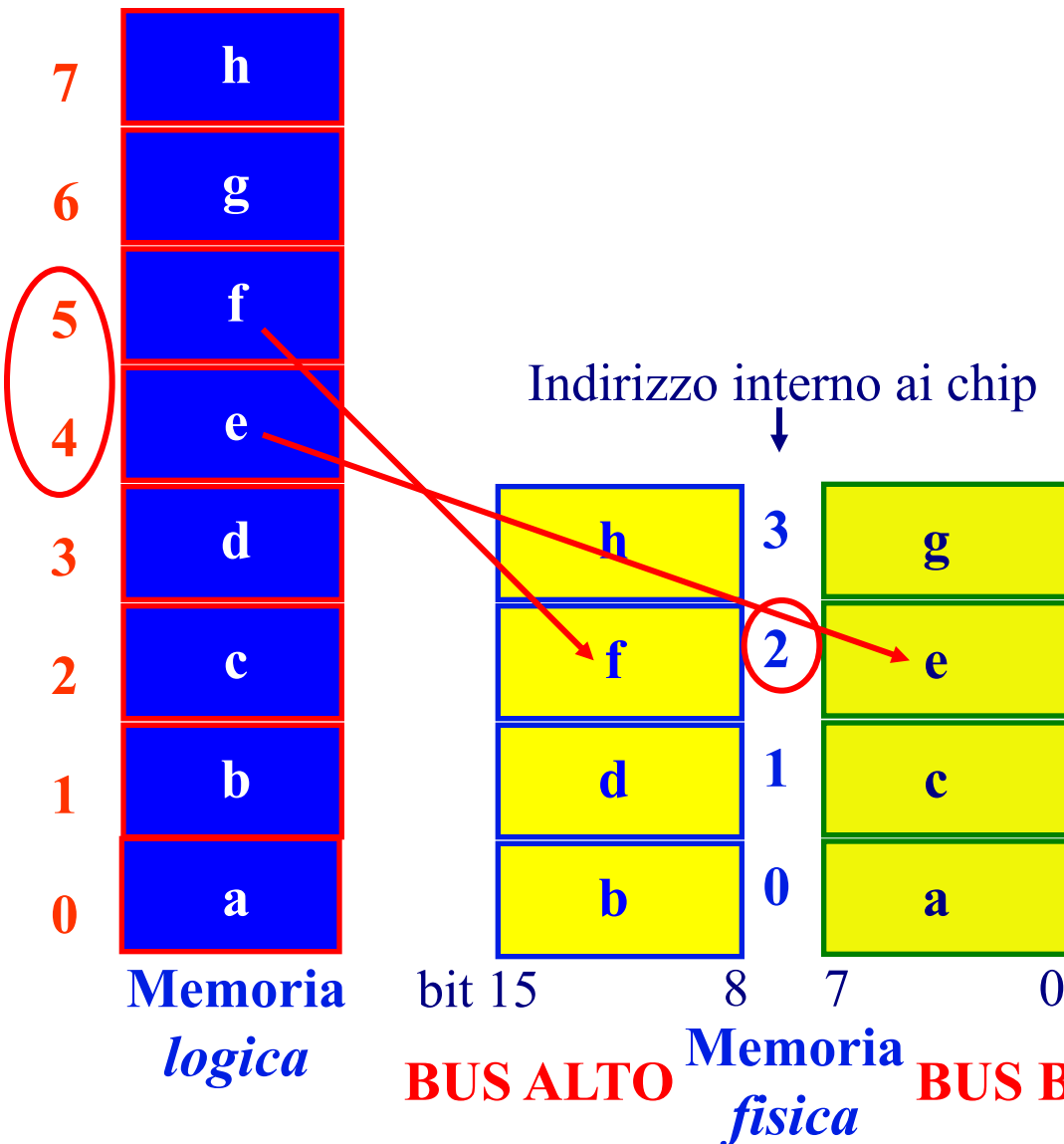


Caso 4 edifici:



Memoria con processori a parallelismo > 8

Il caso dei 16 bit



Indirizzo fisico memorie = Indirizzo logico / 2

Sul piedino **A0** della memoria -> **BA1** bus

A1 della memoria -> **BA2** bus

.....

Word(3) -> Byte_h(1) e Byte_b(2) -> 2 letture

↑
Logico
(d ,e)

↑
Fisico
(d)

↑
Fisico
(e)

(Lettura bytes 3 e 4 che però stanno a indirizzi fisici interni delle memorie **differenti**)

*Le memorie fisiche vanno sempre in coppia
Per ogni "banco" ci deve essere un ByteEnable
BE0 per banco 7-0 e BE1 per banco 15-8*

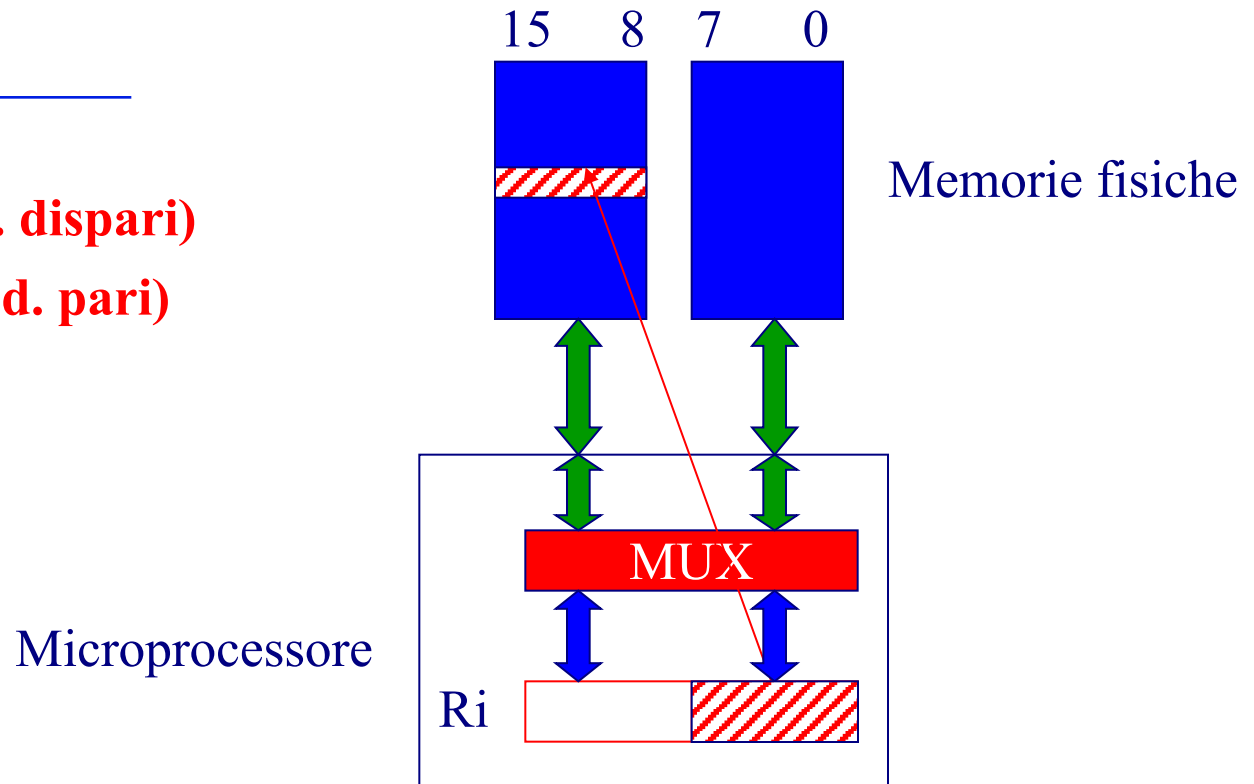
Memorie sempre in coppia

Ad esempio 2 x 8K = 16 K

Memorie con bus a 16 bit

Lo scambio byte alto esterno, byte basso del registro e viceversa avviene *all'interno* del microprocessore

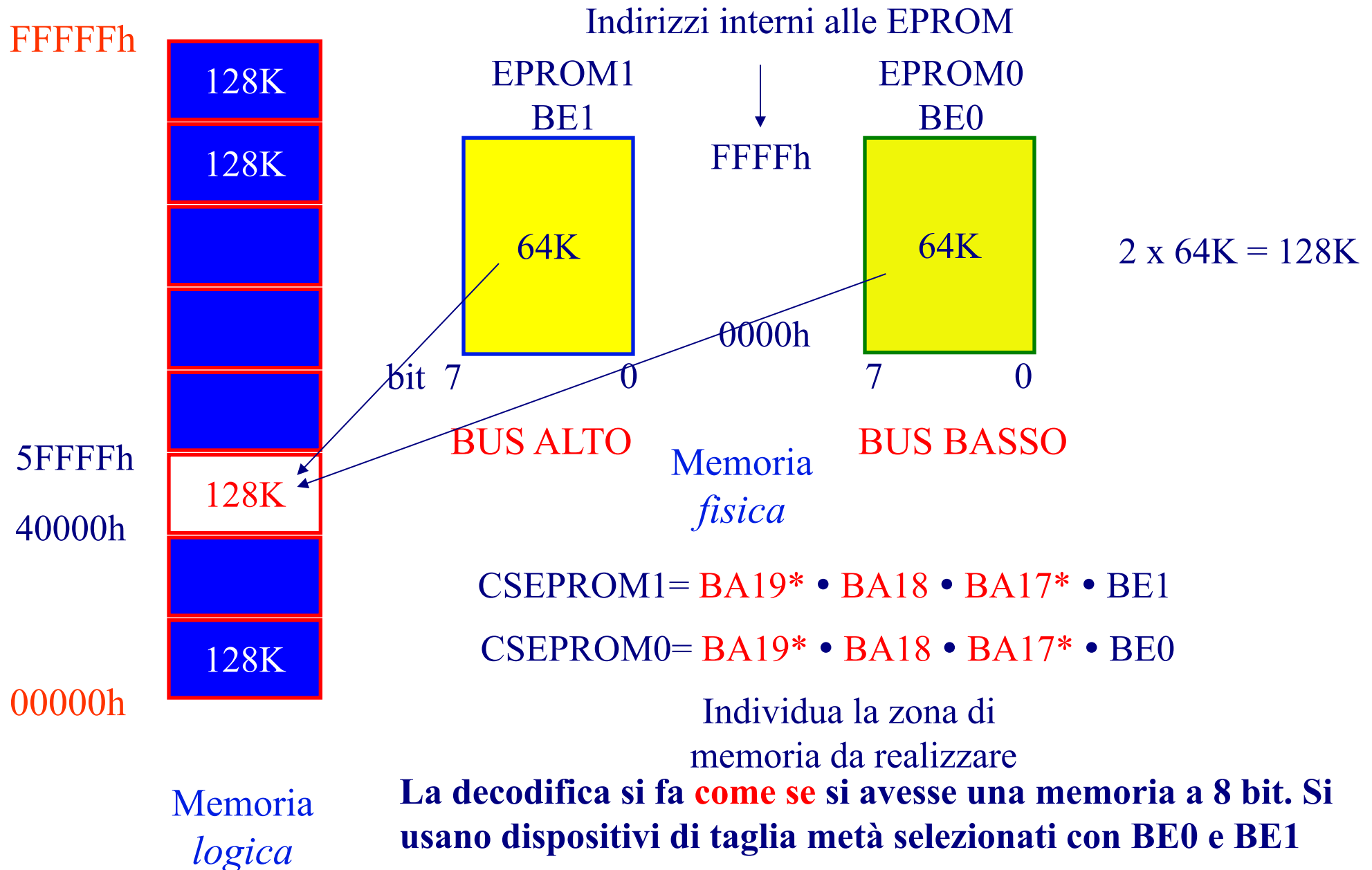
BE1	BE0	
1	1	Word
1	0	Byte alto (ind. dispari)
0	1	Byte basso (ind. pari)
0	0	Non possibile



BA0 del processore non viene generato (di fatto seleziona il banco - al suo posto **BE0** e **BE1**)
BA1 del processore connesso ai piedini **A0** delle memorie
BA2 del processore connesso ai piedini **A1** delle memorie
etc. etc.

Memoria con processori a parallelismo > 8

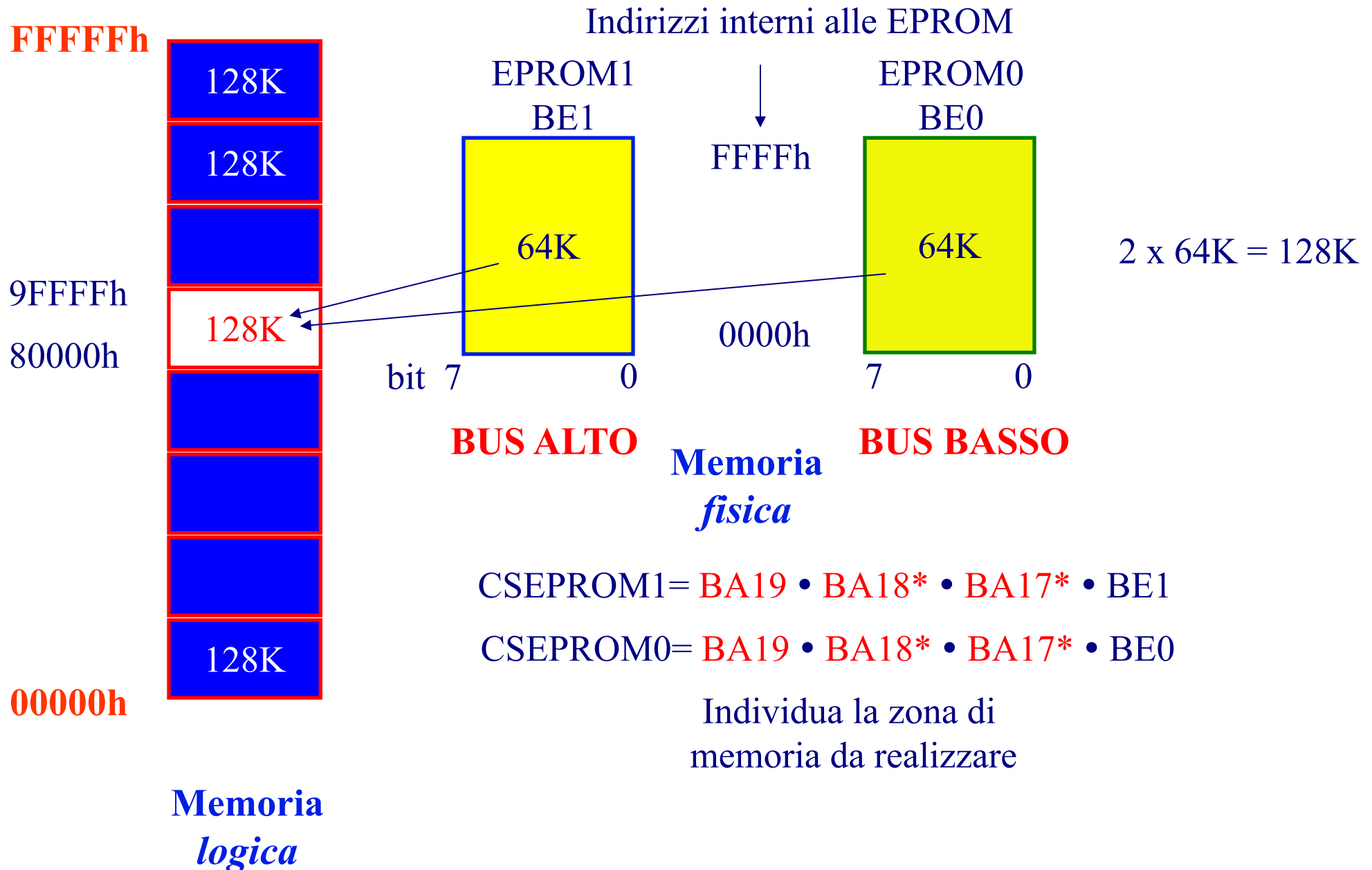
Il caso dei 16 bit



Le memorie vanno sempre in coppia (16 bit)

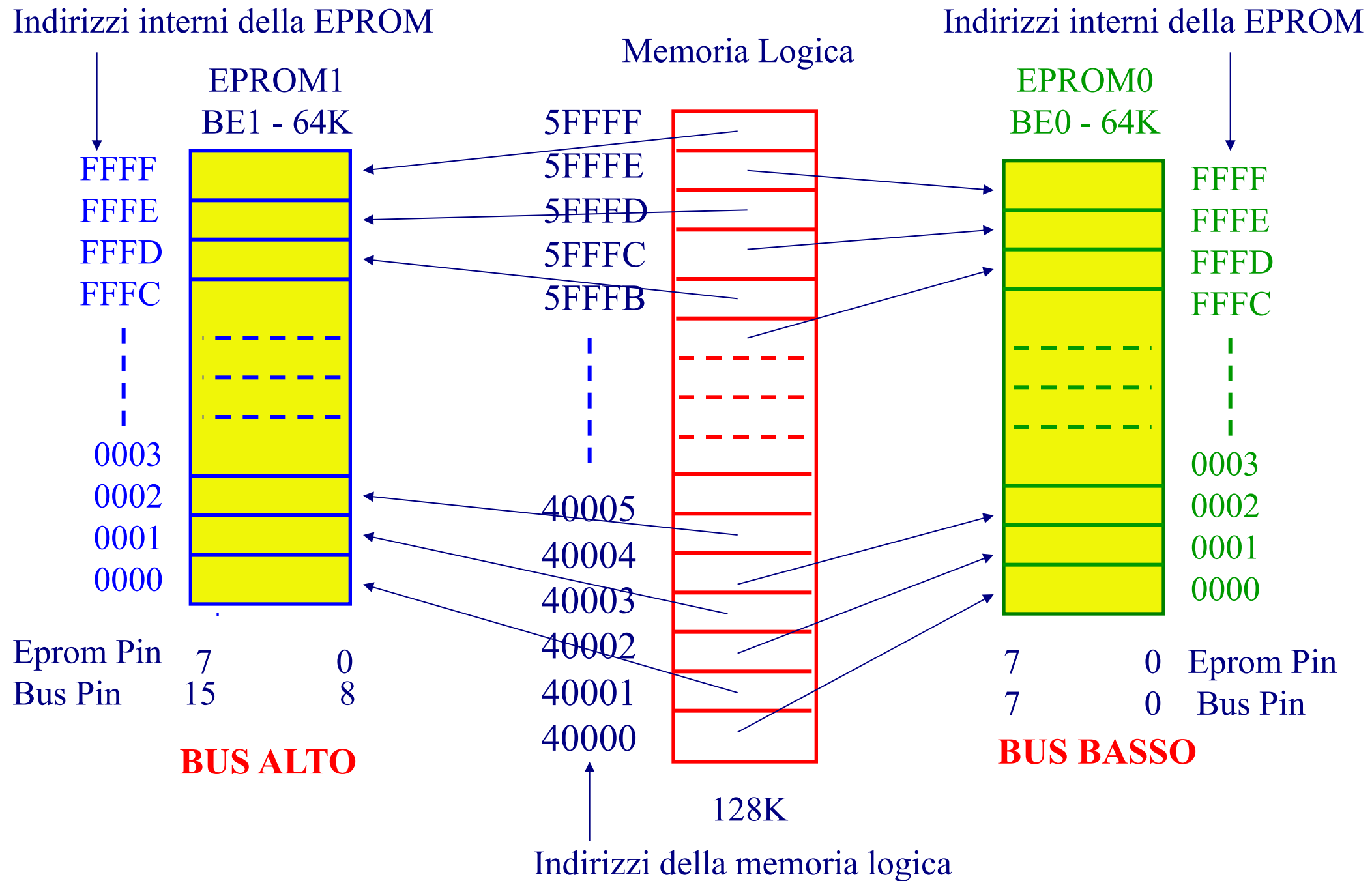
Memoria con processori a parallelismo > 8

Il caso dei 16 bit



Memoria con processori a parallelismo > 8

Il caso dei 16 bit

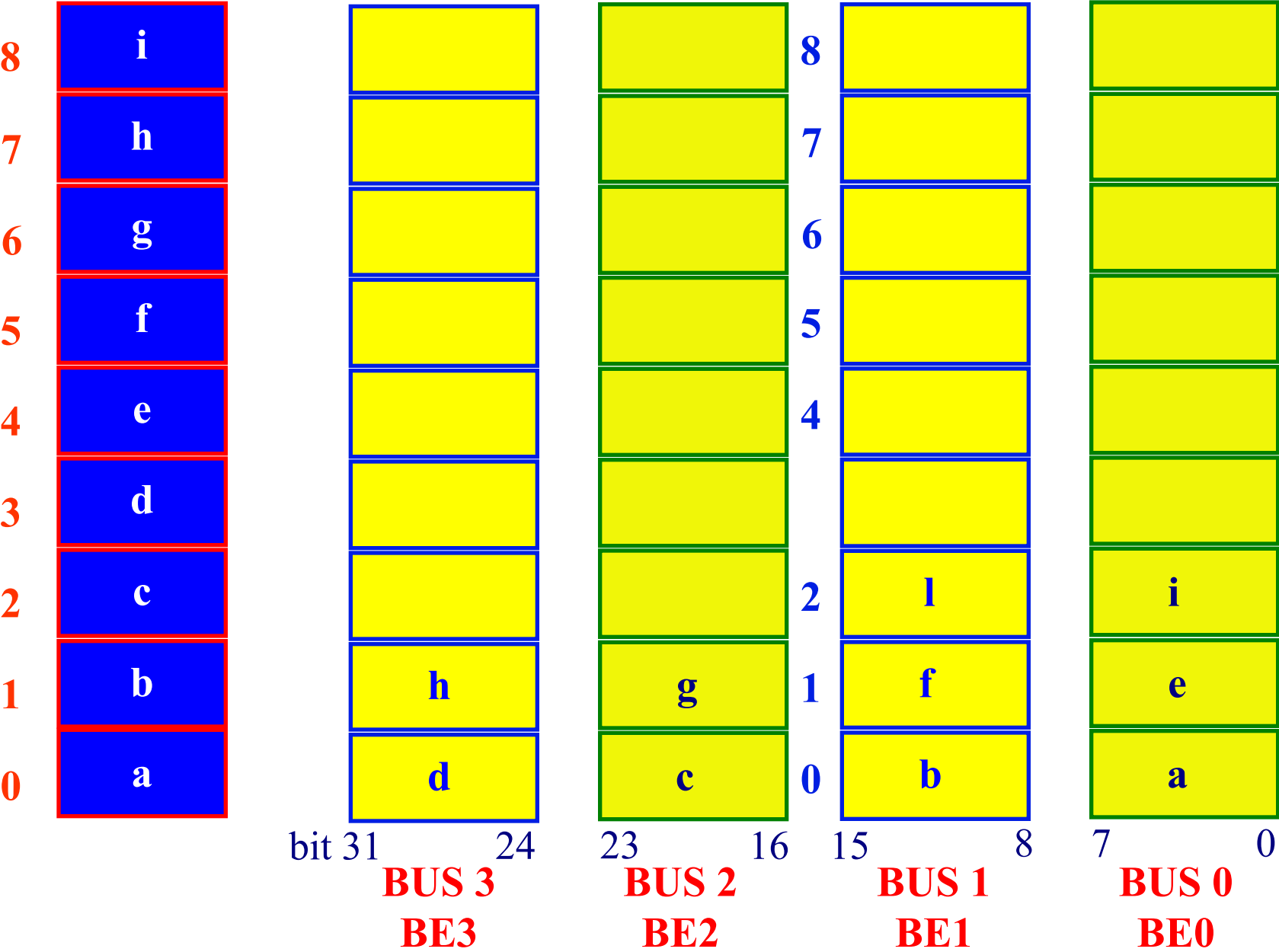


Memoria con processori a parallelismo 32 bit

Memoria
logica

Memoria
fisica

Indirizzo fisico =
Indirizzo logico/4



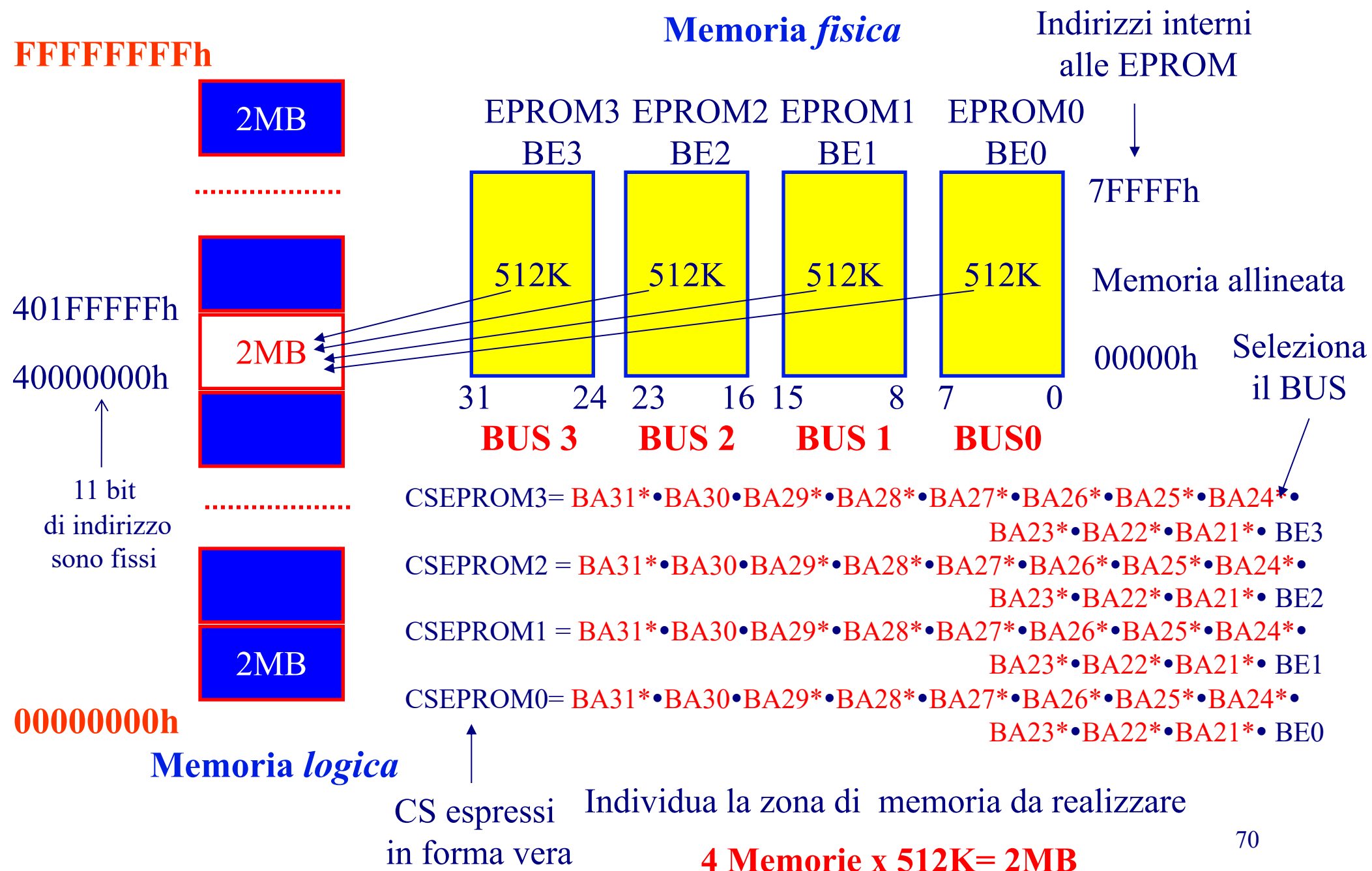
Bus enable con parallelismo 32 bit

BE3	BE2	BE1	BE0		
1	1	1	1	Word 32 bit	Lo scambio fra i bytes (half word) dei banchi di memoria e i byte (half word) dei registri e viceversa avviene <i>all'interno</i> del microprocessore
0	0	1	1	Half word bassa	
1	1	0	0	Half word alta	
0	0	0	1	Byte 0-7	
0	0	1	0	Byte 15-8	
.....					
<i>etc.</i>					

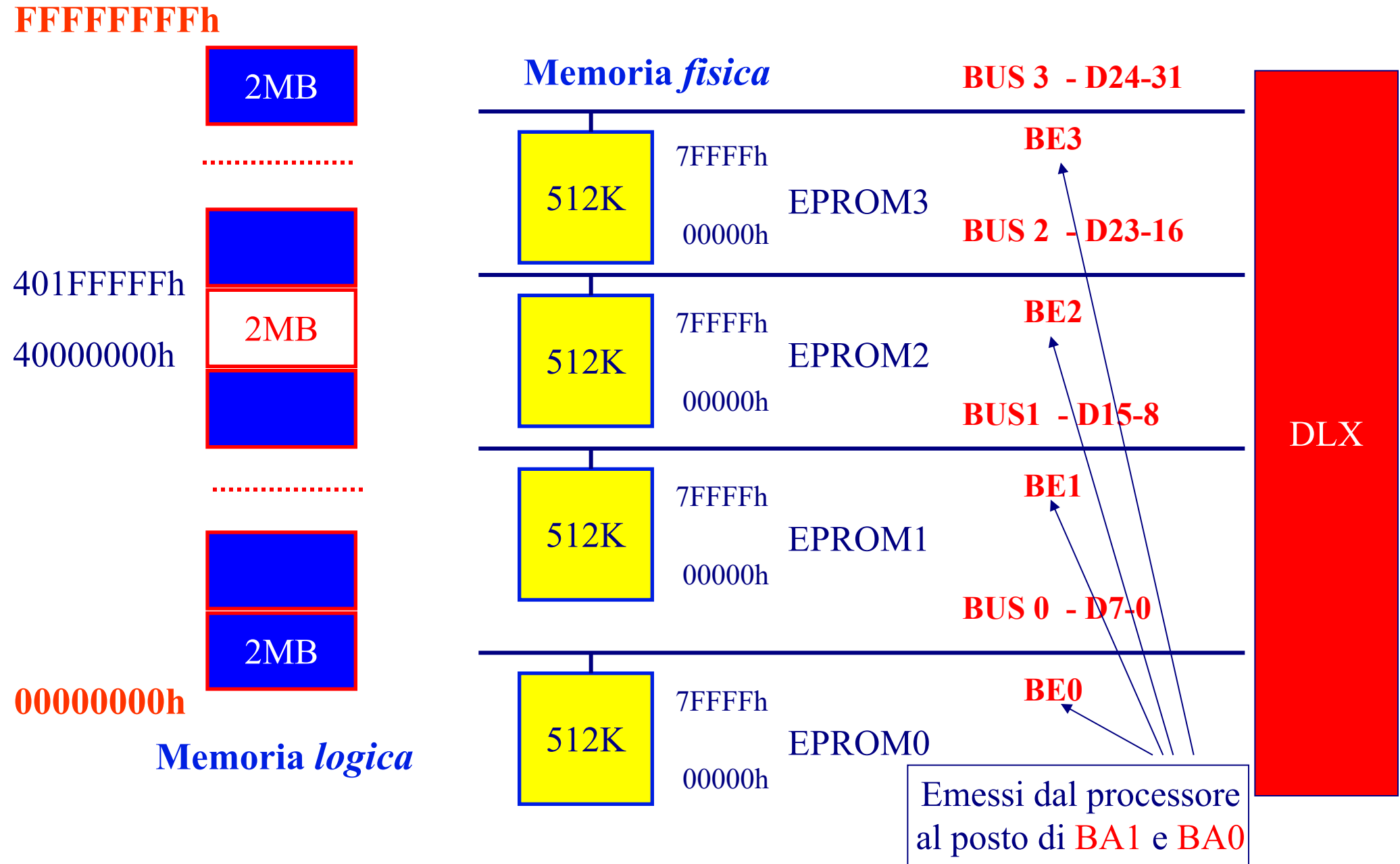
N.B. BA0 e BA1 del processore non vengono generati (di fatto selezionano uno dei banchi - al loro posto BE0, BE1, BE2, BE3)

BA2 del processore connesso ai piedini A0 delle memorie
BA3 del processore connesso ai piedini A1 delle memorie
etc. etc.

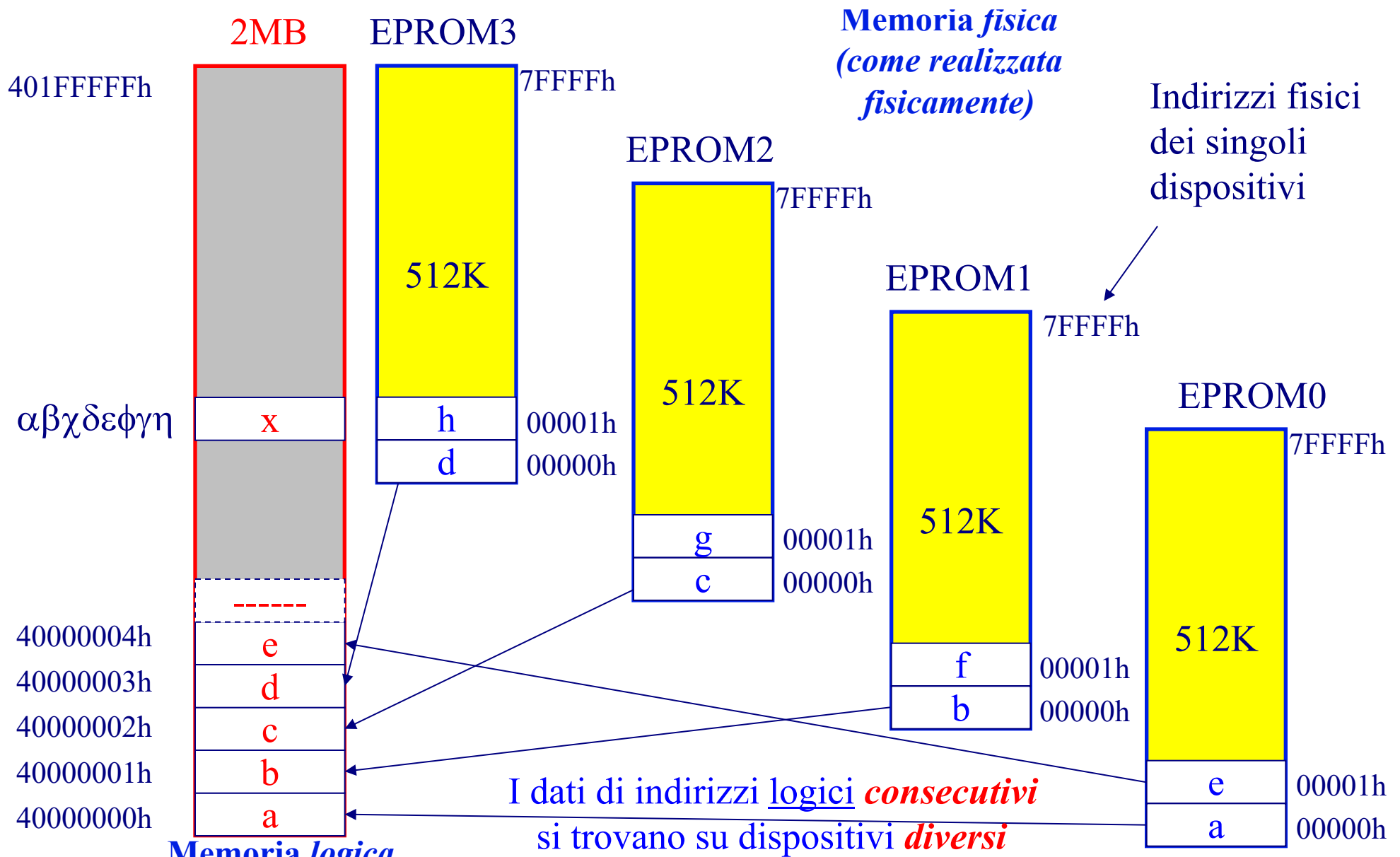
Memorie con parallelismo 32 bit



Memorie con parallelismo 32 bit



Memorie con parallelismo 32 bit



**Memoria logica
(come vista dal
programmatore)**

*La cella x di indirizzo logico αβχδεφγη si troverà all'indirizzo fisico
 $i = \alpha\beta\chi\delta\epsilon\phi\gamma\eta / 4$ del dispositivo EPROMi ove i è il resto della divisione*